

计嵌试卷真题 答案及解析

GPT 6 Astra Generation

2023 • 2025 • 2026

计算机原理与嵌入式系统

全部 148 题，含 72 个设计与综合题子问。
逐题答案、计算推导、代码分析与接线示意图。

依据逐页校核后的《计嵌试卷真题集》独立求解。
遇到原卷矛盾、条件缺失或单选题多解，保留原题并说明成立条件。
本册为学习解析，不是任课教师发布的标准答案。

生成日期：2026 年 9 月 21 日

目录

使用说明	ii
第一章 2022–2023 学年试卷答案及解析	1
第二章 2024–2025 学年：答案及解析	17
第三章 2026 年试卷答案及解析	34
核验资料	49

使用说明

本册按 2023、2025、2026 年顺序作答，对应真题集的每年独立题号。2023 年共 49 题，2025 年共 50 题，2026 年共 49 题；设计与综合题的所有备选题均作答。每章先列答案速查，再给逐题推导。

组合选择题的判定。先分别判断各陈述，再检查选项中所列陈述是否全部满足题问。原题若只问“正确的有”而未要求列全，不擅自把“正确陈述的子集”判错；若由此出现多个可选项，直接报告多解。对于无可选项、地址范围倒置、硬件条件与代码不一致等问题，在该题正文写明。

计算约定。存储容量按二进制数量级计算（1 KiB = 1024 B；题文沿用 kB/MB 时亦按此口径）。C 代码按 STM32 常用 ARM 32 位数据模型、小端序分析；有符号溢出的硬件标志与 C 语言的未定义行为分别说明。Cache 命中率统计题目指定的数组数据访问，不把寄存器变量、取指令等另计入分母。

原题与可运行程序。先回答题文与给定代码的实际行为，再提供必要的修正。修正版明确列出前提，不代表原代码已经能编译或已在实验箱运行。芯片引脚或寄存器的判断以 ARM、ST 官方资料为依据，关键处附引用标记，完整出处见卷末。

图示。存储扩展题同时区分题设的逻辑总线与 STM32F103ZE 的真实 FSMC 能力。可编辑的接线图源文件保存在项目的 `assets/answers-astra/` 中；本册嵌入矢量导出图，便于放大查看。

第一章 2022–2023 学年试卷答案及解析

本章按校核后的原卷解答全部 49 题，包括综合题的全部 28 个小问。选择题先判断各陈述，再核对原卷选项；组合选项中所列项目全部符合题干即可，不要求列尽所有正确或错误陈述。原卷有歧义时分别给出课程常用口径和严格解释，不把猜测写成唯一答案。

答案速查

题号	填空答案
1	运算器；控制器
2	控制相关（控制冒险）；分支代价（分支延迟）
3	状态寄存器；控制寄存器
4	断点（返回地址）；现场（上下文）
5	参考填写 Keil MDK / μ Vision；J-Link，须以本课程实验配置核实

题号	答案	题号	答案
6	F	18	F
7	C	19	C
8	E	20	E
9	B	21	E
10	B；宽泛定义下 B/C/D/E	22	B（存储区条件见解析）
11	F	23	C
12	C	24	E
13	A	25	D
14	B（VI 的资料限制见解析）	26	C、D 均成立，不是唯一单选
15	A	27	C
16	A	28	C
17	B	29	A

题号	30	31	32	33	34	35	36
答案	×	×	✓	✓	✓	×	×
题号	37	38	39	40	41	42	43
答案	✓	×	✓	✓	×	✓	×

第 33 题的“通道”按读写数据线分设理解；第 40 题按 Cortex-R 系列具有锁步可靠性技术理解，具体限定见逐题解析。

一、填空题

第 1 题 CPU 组成：运算器；控制器

五大基本部件为运算器、控制器、存储器、输入设备、输出设备。CPU 的核心职能是运算和控制，所以两空填运算器、控制器。现代处理器还可能集成 Cache 等部件，但本题要求的是五部件模型中的主要两项。

第 2 题 流水线分支：控制相关；分支代价

分支方向或目标尚未确定时，后续指令的正确取指地址不确定，形成控制相关，也称控制冒险或控制冲突。若已取入错误路径指令，需要将其冲刷并重新填充流水线，损失的时间称为分支代价（branch penalty），也可称分支延迟。这里不是数据相关，也不是“延迟槽”的定义。

第 3 题 I/O 接口：状态寄存器；控制寄存器

状态寄存器报告就绪、忙、完成或错误等状态；控制寄存器接收工作方式、启动命令和中断使能等设置。它们与题目已给的输入、输出数据缓冲寄存器共同构成典型 I/O 接口。“命令寄存器”可作为第二空的相近术语。

第 4 题 异常现场：断点；现场

按本题“下一条待执行指令”语境，该地址称为断点或异常返回地址；寄存器等运行环境称为现场或上下文。Cortex-M3 异常入口会硬件保存 R0–R3、R12、LR、返回 PC、xPSR，软件按需保存其余寄存器。故“断点”只是现场的一部分，不能把两空都填为“现场”。同步故障的返回地址还可能指向导致故障的指令，须结合异常类型理解。^[ARM-M3]

第 5 题 实验工具：参考 Keil MDK / μ Vision；J-Link

Keil MDK 是常见的 Arm 微控制器开发工具套件，其 IDE 为 μ Vision；J-Link 是可用于下载与调试的硬件探针。若本课程采用这套组合，可据此填写。

这是一道依赖课堂实验事实的题。试卷没有给出 IDE 界面、实验指导书或探针型号，故无法仅凭题面排除 IAR、ST-LINK、ULINK 等其他配置。“可进行交叉编译”也不能唯一推出 Keil。上述为参考填写，两个品牌均不是由题目独立证明的唯一答案。

二、单项选择题

第 6 题 进制比较：F；正确陈述为 II、VI

先统一成十进制：

$$2023H = 2 \cdot 16^3 + 2 \cdot 16 + 3 = 8227, \quad 2F21H = 12065, \quad 2049D = 2049.$$

因此 $12065 > 8227 > 2049$ 。I 错：2023H 不是最大；II 对：二进制数最大；III 错：2049D 不是最大；IV 错：2023H 不是最小；V 错：二进制数不是最小；VI 对：2049D 最小。

原卷 A=I、V, B=II、V, C=III、IV, D=III、V, E=IV、V, 每项都含错误陈述, 故选 F。尤其 B 原文是“II、V”, 不是“II、VI”, 不能把选项改成正确组合后再选 B。

第 7 题 补码溢出: C (II、III、IV)

16 位补码范围为 $[-32768, 32767]$, 解码得 $a = 32766$ 、 $b = 2$ 、 $c = -2$ 、 $d = -32767$ 。

陈述	精确数学结果	是否越界	陈述真假
I	$a + b = 32768$	正溢出	错
II	$a + c = 32764$	不溢出	对
III	$b + c = 0$	不溢出	对
IV	$b + d = -32765$	不溢出	对
V	$c + d = -32769$	负溢出	错

只有 C 所列 II、III、IV 全部正确; 其他 A-E 选项均含 I 或 V。判断补码溢出应比较真实结果和有符号范围, 不能把最高位进位直接当成溢出。

第 8 题 IEEE 754: E, -2^{97}

把 F0000000H 分成符号、阶码和尾数字段:

$$S = 1, \quad e = (11100000)_2 = 224, \quad f = 0.$$

这是正规数, 因此 $x = (-1)^1(1 + 0)2^{224-127} = -2^{97}$ 。A、B 的指数 27 错; C、D 的指数 96 错; F 的符号错, 故 E。

第 9 题 微程序控制: B (I、III)

I 对: 在微程序控制器中, 机器指令由对应微程序解释; II 错: 一般并非恰好一条微指令; III 对: 执行过程可分解为一系列微操作; IV 错: 微指令不是由一条机器指令来执行; V 错: 一般程序不能由一条微指令执行; VI 错: 微指令编码微操作控制及后继地址等信息, 并非若干机器指令的集合。故 B 全真, 其余选项均含假项。

第 10 题 流水线吞吐: 简化模型下 B; 宽泛定义下 B、C、D、E

I 错: 性能改善通常使平均 CPI 降低, “提高 CPI” 不是流水线的目标。II 对: 重叠执行可提高 IPC。III 对: Load/Store 将访存集中在专门指令中, 便于规则分段。IV 按典型教材比较为对: 规则的 RISC 指令组织通常更便于流水化; 这不表示 CISC 不能高效流水。VI 对: 超标量可通过多发射及多个执行通路提高吞吐。

V 的判断取决于“标量”和“多条流水线”的定义。若采用把标量视为单发射、把多条流水线特指每周期并行发射多条指令的简化模型, 则 V 判错, 只有 B (II、III、IV) 全真。

从微体系结构的严格定义看, 单发射处理器也可有整数、浮点、访存等不同执行流水线; “标量”还可与“向量”相对。因此 V 的字面表述不能无条件判假。若允许上述多执行流水线含义, V 也真, 则 B、C、D、E 均为全真组合, 题目失去唯一性。题面不能证明本课程采用哪一种定义, 故不把简化模型下的 B 冒充成已知的课程标准答案。

第 11 题 五级流水：F；I、V 正确

I 对：不同指令可处于不同流水段而重叠执行。II 错：各阶段不必共用同一功能部件，共用还可能造成结构冲突。III 错：各段延迟应尽量均衡，原文明确是“最好不相等”。IV 错：最短段不能保证其他段完成。V 按统一时钟周期的题意为对，理想情况下 $T = \max_i t_i$ ，实际还需加流水寄存器及时钟裕量。VI 错：平均值可能短于最慢段。

正确集合为 I、V，原卷 A–E 没有全真组合，故 F。严格说各组合逻辑段的固有延迟不会被时钟强行变成相同值；被最慢段限制的是所有段共用的节拍。

第 12 题 RAW 数据相关：C (II、III、V)

I 无相关：第一条写 R0，第二条只读 R4、R5 并写 R3。II 有相关：第二条读取第一条产生的 R0。III 有相关：ADD 读取 LDR 刚产生的 R1，属于典型 load-use 相关。IV 无相关：两条涉及的寄存器互不依赖。V 有相关：LDR 的访存地址 R0 来自前条 ADD。VI 无相关：两条涉及的寄存器互不依赖。

因此 C 全部属于数据相关；其余组合含无关项。转发或停顿可以处理相关，但不会消除程序本身的生产者–消费者依赖关系。简单按序五级流水重点是 RAW，不能把所有“出现过相同寄存器”都等同为必须停顿。

第 13 题 Cache 的作用：A

高速小容量 Cache 利用时间和空间局部性，减少处理器等待慢速主存的次数，主要解决 CPU 与主存速度不匹配。B、C 涉及辅存层次；D 是外设速度；E、F 是容量问题，均不是本题 Cache–主存结构的直接目标。

第 14 题 Cache 映射：可确认的答案 B (III、IV)

I 一般为对：同容量、同块大小且替换合理时，全相联消除了直接映射的冲突限制，命中率通常更高，不是对任意程序和任意替换策略的绝对定理。II 错：通常不是直接映射比组相联命中率更高。III 对：直接映射每次只需检查确定的一行，硬件较简单。IV 对：组内路数通常取较小值，例如 2、4、8、16。V 错：组数取决于总容量、块大小和路数，不以“小路数”推出“小组数”。VII 一般为对：PC 处理器的各级缓存普遍采用组相联组织。

VI 不能仅凭已公开的 RM0090 说明确认为真。STM32F407 的 Flash ART 确实有指令缓存和供 Flash 数据读取使用的数据缓存；不能用“Cortex-M4 内核无通用 Cache”推出芯片没有任何缓存。RM0090 描述了缓存行及替换行为，但这些信息本身不能证明其一定是题目所称的组相联。^[RM0090]

B 中 III、IV 均可确认；A 含 II，C、D 含 V，不成立。E 依赖 VI：若课程另有可核实资料证明 VI 为真，E 也会成立，原卷将不再是唯一单选。当前资料下采用可确认的 B，并保留 VI 的证据限制，而不虚构 ART 的相联度。

第 15 题 半导体存储：A (I、II、III)

I 对：半导体存储具有小型化、轻量化优势。II 对：片内 Flash/SRAM、主存 DRAM、外存 SSD 都使用该技术。III 对：U 盘的主要存储介质为闪存。IV 错：HDD 的主要大容量介质是磁性盘片；其控制器中可能有半导体缓存，但这不能把 HDD 的介质归为半导体存储。V 对：现代内存条主要使用 DRAM。VI 对：STM32F407 内含半导体 Flash 和 SRAM。

所以 I、II、III、V、VI 真，只有 IV 假。A 全真；B–E 均含 IV，故 A。

第 16 题 辅存接口：A (I、II、III)

I 对：早期 PC 常见 IDE/ATA 硬盘接口。II 对：SATA 可连接 HDD。III 对：SSD 也有 SATA 型号。IV 错：普通 MCU 系统一般不采用 SATA 作为常规硬盘接口。V 对：MCU 可经 SPI 连接串行 Flash 等存储器。VI 对：MCU 可经 I²C 连接 EEPROM 等存储器。

A 所列前三项全真；B–E 都包含错误的 IV。这里按试卷年代和其“往往 / 可以”的限定判断，并没有断言所有 SSD 都使用 SATA。

第 17 题 总线与仲裁：B (I、II、IV)

I 对：总线把部件接到公共通信通道。II 对：通常区分发起传输的主设备和响应的从设备。III 错：异步总线无需所有设备共用统一时钟；同步总线的总线接口也不要求设备所有内部逻辑都同频。IV 对：多个主设备竞争共享资源才需要主设备间总线仲裁。V 错：只有一个主设备时不存在主设备之间的竞争。VI 对：单主总线可仅包含寻址、数据传输两类阶段。

B 全真，A、C 含 III，D、E 含 V，故 B。这里“仲裁”特指题目中的总线主控权仲裁，不包括从设备内部的其他资源调度。

第 18 题 地址空间：F；不正确的只有 IV

I 对：并行地址信号线表示地址的不同二进制位。II 对：一般计算机按字节编址。III 对：地址标识存储单元。IV 错：地址也能寻址 I/O 端口寄存器，STM32 即采用存储器映射 I/O。V 对：通常 Cache 对程序员透明，同一主存地址通过 Tag、Index、Offset 查找其缓存副本，不产生另一个可任意寻址的 CPU 数据空间。VI 对：虚拟地址经地址转换形成物理地址。

题目问“不正确”，A–E 的每个组合均至少含一条真命题，因此 F。D 原文为“IV、V”，不是只列 IV；不能删掉 V 后选 D。Cache 内部行号和主存物理地址编码不同，并不意味着它们属于两个独立的程序员地址空间。

第 19 题 AHB 信号：C (II、III 不正确)

I 对：完整 AHB 支持多主设备，AHB-Lite 则是单主接口的简化形式。II 错：HADDR 与数据线分设。III 错：读 HRDATA 与写 HWDATA 分设。IV 错：AHB 使用 HCLK 同步时序。V 对：一个传输的数据阶段可与下一个传输的地址阶段重叠。VI 按“读写数据线分设”理解为对。^[AHB]

题目要求组合中项目均“不正确”，C 的 II、III 满足；其他 A–E 组合都含真项。虽 IV 也错，C 无需把所有错项列尽。AHB 的读写数据线分设不等于 AXI 式读写事务完全独立并行。

第 20 题 AHB 突发：E (III、VI 不正确)

I 若“块长度”指整个 burst 的拍数或总字节数，则错：HSIZE 给每拍字节数，HBURST 描述拍数 / 突发类型。II 对：HTRANS 表示 IDLE、BUSY、NONSEQ、SEQ。III 错：INCR 支持未指定长度的递增突发。IV 对：可使用递增或回绕地址。V 对：连续传输及流水重叠减少开销。VI 错：WRAP4 表示 4 拍，每拍不必是 32 位字；例如 4 个半字只传 8B。^[AHB]

E 的 III、VI 全错；A、B 含正确的 II，C、D 含正确的 IV，故 E。若把 I 的“数据块”

仅理解为一拍数据的宽度，I 的表述可成立，但仍不改变 E 的选择。

第 21 题 异常向量表：E (III、V、VI)

I 错：复位默认向量基址位于 `0x00000000`，属于架构的 CODE 区。II 错：VTOR 可把向量表移至满足对齐要求的可读存储器，包括 SRAM。III 对：默认基址属于 CODE 区；底层具体映射哪种物理存储器由芯片启动配置决定。IV 错：表中不是跳转指令。V 按异常向量条目为对：存放处理程序入口；第 0 项是初始 MSP 这一例外必须区分。VI 对：每项 32 bit，即 4B。[ARM-M3]

只有 E 不含 I、II、IV。不能把“CODE 地址区”误解为只能由物理 Flash 实现，也不能把表首项当作 Reset 向量。

第 22 题 字对齐与存储区：通常选 B；只看对齐则 A、B 都满足

按 Arm 常用 ABI，`unsigned int` 为 4B、自然对齐为 4B，故地址须满足 $A \bmod 4 = 0$ 。[AAPCS32]

项目	起始地址	4B 对齐	架构区域 / 说明
I	<code>0x00000000</code>	是	CODE；通常是启动映射
II	<code>0x20000000</code>	是	SRAM 区
III	<code>0xFF000000</code>	是	系统 / 厂商定义区域，非普通 SRAM
IV	<code>0x20220004</code>	是	SRAM 区
V	<code>0x20230002</code>	否	低两位为 10
VI	<code>0x20230008</code>	是	SRAM 区

若题意为“普通临时变量置于可用 SRAM，且正确对齐”，则 II、IV、VI 符合该区域模型，选 B；I、III 不能据题面确认可写，V 不对齐。

题目只说“某基于 Cortex-M4 的计算机”，未给实际 RAM 布局。处于 SRAM 区不代表这些高地址一定有实体 RAM；CODE 区也可能映射 RAM。若只考数值对齐，I、II、III、IV、VI 都对齐，A 和 B 均满足。B 是常用的存储区假设下的答案，不应以对齐规则为由宣称 I、III 也不对齐。

第 23 题 M4 模式与权限：C (II、III、IV)

I 错：复位后处于特权级。II 对：复位后选用 MSP。III 对：处理器有 Thumb 执行状态与调试状态；没有可切入的经典 ARM 指令状态。IV 对：运行模式为 Thread 和 Handler。V 错：非特权访问受架构区域权限及可能存在的 MPU 约束。VI 错：非特权代码不能任意读写所有特殊寄存器及系统控制寄存器。[ARM-M4]

C 全真，A、B 含 I，D 含 V，E 含 VI，故 C。模式、状态和访问等级是三个不同概念。

第 24 题 M3/M4 存储系统：E (III、V、VI)

I 在处理器集成选项层面成立：架构可实现小端或字节不变大端；不能理解成任意 STM32 都可由普通软件动态切换端序。II 错：外部 RAM 在存储器属性允许执行时也能存放和执行代码。III 对：32 位地址对应 4GB 线性空间。IV 对：内核的写缓冲可在满足存储器属性和顺序要求时减少等待，并非每次写都保证加速。V 对：所列 `0x00000000 - 0x0000FFFF` 包含在 CODE 区内。VI 对：访问须服从区域的类型、权限和属性。[ARM-M3][ARM-M4]

A-D 都含错误的 II; E 全真, 因此端序集成细节不影响 E 的选择。

第 25 题 位带区域: D (IV、VI)

I 错: CODE 区不是标准位带目标区。II 对: 片上外设 $0x40000000-0x400FFFFF$ 对应 32MB 别名区。III 错: 片外外设区不是该位带目标区。IV 对: SRAM $0x20000000-0x200FFFFF$ 也有对应别名区。V 错: 外部 RAM 区不是标准位带目标区。VI 对: 对别名字进行一次访问可原子地修改对应位, 免去软件的读-改-写窗口。[ARM-M3]

因此真项为 II、IV、VI, 只有 D 所列 IV、VI 全真。位带须由具体实现支持, 且不能把这一结论扩展为对任意有副作用的外设寄存器都能随意操作。原卷 IV、V 的“位段带名”按其明显所指理解为“位带别名”。

第 26 题 栈与权限: C、D 均符合, 原卷并非唯一单选

I 对: MSP 和 PSP 是两个栈指针。II 可按通常初始化要求理解: 使用某栈前需有有效初值; 但严格过程是复位时硬件从向量表第 0 项装载 MSP, PSP 需软件初始化, 软件也可重设 MSP, 不能说二者都必须由初始化指令首次写入。III 错: Thread 模式的栈选择 SPSEL 与特权位 nPRIV 是独立状态, 非特权线程并非架构上禁止使用 MSP; 它只是不能任意修改受特权限制的配置。IV 错: 满递减栈入栈使 SP 减小。V 错: 栈也可放在正确配置的外部可写 RAM。VI 错: 并非所有区域都可作为栈, 例如只读 Flash、未实现地址或不合适的设备区域。[ARM-M3]

C=III、IV、V 与 D=IV、V、VI 都完全由错误陈述组成, 均满足题干“不正确”。因此至少 C、D 同时成立, 不能以单选格式强行只给 D。若将 II 严格读成“MSP 和 PSP 均由初始化程序首次装载”的必然命题, II 也假, B 也会满足; 此为原卷 II 措辞的另一处不严谨。

第 27 题 F103 时钟: C (II、V 不正确)

I 对: 主 PLL 的输入不取 LSE、LSI。II 错: HSI 是片内高速 RC, 只有 HSE 来自外部晶振或外部时钟输入。III 对: SYSCLK 可取内部 HSI、外部 HSE 或 PLL。IV 按 TIM 通用 / 高级 / 基本定时器为对: 内部计数时钟来自 APB 时钟树, APB 预分频不为 1 时还可能先倍频。V 错: IWDG 独立看门狗由 LSI 驱动。VI 对: GPIO 的 APB2 时钟由系统时钟树派生。[RM0008]

C 中 II、V 均错, 其余 A-E 组合含真项, 故 C。IV 的“各个定时器”按课程通常指 TIM 外设; 若无限扩大为 RTC、看门狗等所有定时装置, 则该概括不成立。

第 28 题 端口地址与器件地址: C (II、IV、V)

I 对: 体系结构可以统一编址或独立编址 I/O。II 对: STM32 的外设寄存器映射在确定的总线地址。III 错: USART1 的 SR 地址也是 I/O 端口地址。IV 对: GPIOA 的 ODR 地址是 I/O 端口地址。V 对: I²C 总线地址标识总线器件。VI 错: 器件内部寄存器偏移需按器件协议另行传送, 不能与总线器件地址混同。

真项 I、II、IV、V, C 全真; A、B、D 含 III, E 含 VI, 故 C。

第 29 题 F103 USART: A (I、II)

I 对: USART 支持异步和同步模式。II 对: 可通过接收、发送或错误事件请求中断。III 错: USART 中断优先级可由 NVIC 配置。IV 错: 通信前需按所用引脚配置 GPIO / 复用功能; 默认映射不等于可以忽略引脚模式。V 对: DR 的读侧用于接收、写侧用于发送。VI 错: USART1 接 APB2, USART2 / USART3 接 APB1。^[RM0008]

A 全真; B–E 分别含 III、IV 或 VI, 故 A。

三、判断题

第 30 题 ARM 存储结构: 错

ARM 是处理器 / 指令集家族, 不能把所有型号都归为同一种物理存储组织。许多 Cortex 内核采用改进哈佛结构, 分开指令和数据访问通路, 但程序员仍看到统一地址空间。题目中的“摒弃”和“均”都过于绝对。

第 31 题 地址数据复用: 错

地址线和数据线可以时分复用: 先在公共 AD 引脚上给出地址并锁存, 随后同一组引脚传送数据。是否复用取决于芯片接口设计; 复用以时序和外部锁存等开销换取更少引脚。

第 32 题 Cache 的透明地址空间: 对

在通常透明 Cache 模型中, CPU 对某主存地址的访问可能命中 Cache, 也可能送往主存, 两者指向同一个被访问对象。Cache 内部的行号、标记和偏移用于定位副本, 并不另建一个程序员可见的地址空间。此处与第 18 题 V 的口径一致。

第 33 题 AHB/APB 读写数据线: 按题目语境判对

AHB 使用 HRDATA/HWDATA, APB 使用 PRDATA/PWDATA, 读、写数据确实各有一组信号。因此按本卷把“通道”用于数据信号通路的语境应判对。^[AHB]

这不意味着 AHB 或 APB 具有 AXI 那样分别握手、独立推进的读写事务通道。若将“独立通道”严格定义为后者, 本题表述应判错; 学习时应记住具体信号和握手关系, 而不是混用协议术语。

第 34 题 Cortex-M3 三级流水: 对

三级是取指、译码、执行。三条不同指令的不同阶段可以重叠, 但流水深度并不表示每条指令总是固定三个时钟完成; 访存等待、跳转和部分多周期操作都会影响实际时序。^[ARM-M3]

第 35 题 子优先级与抢占: 错

只有抢占优先级更高的异常才能据优先级规则抢占当前异常。子优先级只决定抢占优先级相同且同时等待的异常先响应哪一个, 单凭子优先级高不能打断正在执行的处理程序。^[ARM-M3]

第 36 题 MSP 可写值：字面命题为错

MSP 可由特权软件写入，架构没有禁止把其值写为零；零也满足其低位对齐约束。但在常规系统中，零不是一个可正常压栈的有效栈顶，随后向下压栈或异常入栈会访问无效位置而引发故障。^[ARM-M3]

因此“不能设置为零”与“不能用零作为有效运行栈顶”不是同一命题，按题面字面判错。若课堂把题目省略的条件理解为“正常工作的栈指针必须指向实际可写栈”，可能意图判对，但那是工程有效性要求，不是 MSP 寄存器写入限制。

第 37 题 异常优先级寄存器访问：对

NVIC 中断优先级寄存器和系统异常优先级寄存器属于特权系统控制寄存器。非特权线程不能直接访问这些寄存器；通常通过特权异常服务完成相关配置。不能把可选的非特权软件触发中断权限误解为可改优先级。^[ARM-M3]

第 38 题 非对齐访问：错

只有部分普通单次加载 / 存储指令支持非对齐。例如 LDM、STM、LDRD、STRD 的非对齐访问会故障；普通 LDR/STR 的非对齐访问还受内存类型及 UNALIGN_TRP 配置约束。“所有访存指令都支持”不成立。^{[ARM-M3][ARM-M4]}

第 39 题 指令周期：对

指令周期是处理器执行一条指令所需的完整过程或时间。它通常包含若干时钟周期，不能与单个时钟周期混同。流水线使多条指令的周期重叠，所以一条指令的执行延迟与稳态平均每条指令的完成间隔也应区分。

第 40 题 Cortex-R 锁步：按系列技术概括判对

双核锁步让两个处理单元执行相同操作并比较输出，从而发现硬件执行分歧。Cortex-R5 等支持这种故障检测手段，服务于高可靠实时系统。^[ARM-R5]

这里判对是指 Cortex-R 系列采用 / 支持这类技术，不表示每个型号、每颗芯片或每个运行配置都必须开启锁步。双核比较可以检测差异，也不应直接等同于自动纠正所有错误。

第 41 题 EXTI 共享向量：错

STM32F407 的 EXTI5–EXTI9 共用 EXTI9_5 中断向量，EXTI10–EXTI15 共用 EXTI15_10 向量。共享 ISR 需要检查对应 pending 位判断哪条线请求中断，因此不是每根线各占一个向量。^[RM0090]

第 42 题 最小硬件系统：对

最小可运行系统需要处理器、供电、必要的时钟和复位条件、启动配置以及程序 / 数据存储资源；某些资源可以已集成在芯片内。SPI、I²C 用于扩展通信，通常不是最小系统必须使用的模块。芯片集成了它们与“最小系统必须包括 / 启用它们”是不同命题。

第 43 题 看门狗最大延时：错

常规定时器位数不能用来推出看门狗计数位数和时钟来源。F103 的 IWDG 使用 LSI、预分频器和 12 位重装载值，近似超时为

$$t_{\text{IWDG}} = \frac{(\text{RLR} + 1)P}{f_{\text{LSI}}}, \quad \text{RLR} \leq 4095, \quad P \leq 256.$$

例如名义 $f_{\text{LSI}} = 40\text{kHz}$ 时，最大约 $4096 \times 256 / 40000 = 26.2144$ 秒；实际 LSI 有频率误差。WWDG 有另一套时钟及计数结构，均不能归结为 65536 个 SYSCLK 周期。[RM0008]

四、设计及综合题（全部作答）

第 44 题 AHB 时序：读传输；B 对应 0，C 对应 4

(1) 传输方向。数据出现在 HRDATA，即读数据通路，故从存储器 / 从设备传往 AHB 主设备（本题为处理器），不是 CPU 经 HWDATA 写向存储器。

(2) HREADY 的作用。在有效时钟采样边沿，HREADY=1 表示当前数据阶段完成，流水传输可以推进；HREADY=0 表示延长数据阶段、插入等待状态，并相应冻结待接受的地址和控制信息。信号边沿附近画出的短折线不应另算成一个完整等待周期。[AHB]

(3) 数据与地址配对。AHB 地址阶段在前、数据阶段在后；结合等待周期，得到

$$B \longleftrightarrow 0x00000000, \quad C \longleftrightarrow 0x00000004.$$

不能把数据线某一瞬间正下方的 HADDR 直接当作该数据的地址。C 出现时地址总线已显示下一次传输的 $0x00000006$ ，但那不是 C 的地址。

(4) 数值的含义。若这是复位时读取默认向量表的序列，地址 0 的 B 是初始 MSP，故 $0x20002000$ 表示初始主栈栈顶；向下入栈会占用它以下的 RAM。地址 4 的 C 是复位异常入口项，题意所指目标程序位于 $0x08000800$ 。[ARM-M3]

向量表中的异常入口必须以 bit0=1 表明 Thumb 状态。故对应上述代码位置的正常原始向量应为 $0x08000801$ 。题面给出的 $0x08000800$ 若真作为向量值，会留下无效的 Thumb 状态，而不是合法的复位入口值。应区分代码地址本身可为偶数与向量所存函数入口值须带 Thumb 标志。

(5) 区域及用途。这些地址位于 Cortex-M3 的 CODE 区 $0x00000000 \sim 0x1FFFFFFF$ 。该区主要用于代码、只读常量及复位 / 异常向量；实际芯片可以把零地址启动别名映射到 Flash 或其他启动存储器。

原图第三个地址确为 $0x00000006$ ，原文无需改为 8。图未给 HSIZE，不能仅因 HRDATA 有 32 根线便断言每拍都传一个 32 位字；若该拍按半字访问，地址 6 是对齐的，若按字访问则不对齐。

第 45 题 SRAM 扩展：按 $1\text{M} \times 16 \text{ bit}$ 解释需 2 片，共 4MiB

(1) 芯片数和题设口径。存储芯片通常按“地址单元数 $\times$ 位宽”标注。将原文含混的“ $1\text{MB} \times 16 \text{ 位}$ ”按 2^{20} 个 16 位单元解释，每片容量

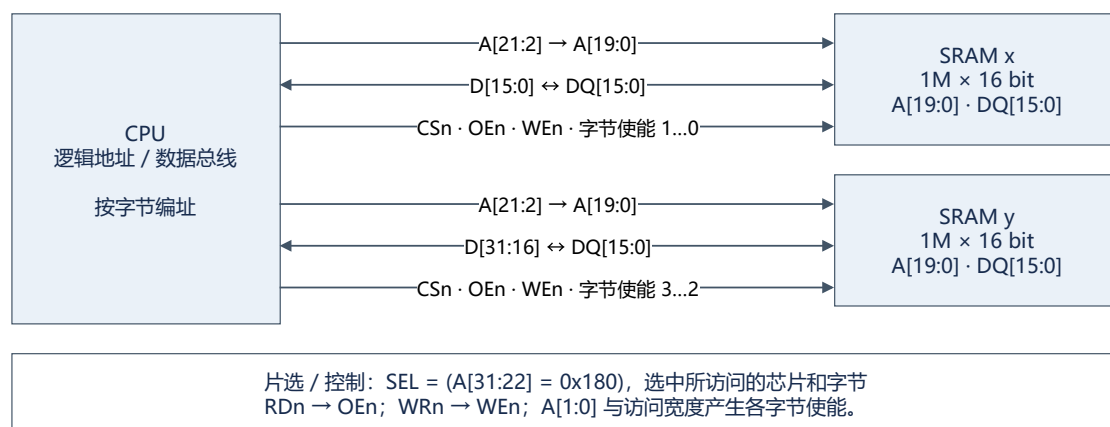
$$2^{20} \times 16 \text{ bit} = 2^{21} \text{ B} = 2\text{MiB}, \quad N = \frac{4\text{MiB}}{2\text{MiB}} = 2.$$

本题 MB 按传统二进制容量使用。若出题人实际想表达“每片总容量 1MB、数据宽度 16 位”，则应为每片 $512\text{K} \times 16\text{ bit}$ 、需要 4 片，接线也须相应增加字扩展。原文不足以完全消除单位歧义；以下采用与其 32 位并联题意相符的 $1\text{M} \times 16\text{ bit}$ 。

(2) 按题设抽象 32 位外部总线画图。两片同步选通，分别组成一个 32 位字的低、高 16 位；由于 CPU 按字节编址，一个 32 位字占 4B，芯片地址应从 CPU 的 A2 开始接。

CPU / 译码信号	SRAM 连接
A[21:2]	并接两片 A[19:0]，共 20 根芯片地址线
D[15:0]	双向连接低半字片的 DQ[15:0]
D[31:16]	双向连接高半字片的 DQ[15:0]
A[31:22] 的全译码	当高 10 位等于 $0\text{x}180$ 时，同时令两片 CS 有效
A[1:0] 与传输宽度	经字节通道逻辑形成字节使能；不接芯片地址输入
读写控制	向两片分别提供相应 OE / WE，图中可按题意略画

2023 年·SRAM 位扩展连接（题设逻辑 32 位总线）



地址范围: $0\text{x}60000000\text{--}0\text{x}603FFFFF$ ；所有芯片共享上图所列地址信号。

图中 $\text{CSn}/\text{OE}/\text{WE}$ 均低有效；各芯片仅在对应字节通道被选中时使能。
 真实 STM32F103ZE 的 FSMC 数据总线最多 16 位，实际接线方案见正文。

真实 STM32F103ZE 的实现限制。32 位内核数据总线不等于芯片向外引出 32 位 SRAM 数据总线。F103 的 FSMC SRAM 接口支持 8 / 16 位外部数据，32 位 AHB 访问可被拆成连续较窄访问。因此不能直接把上图两片的 32 根数据线全接到真实 F103 的 FSMC。
 [RM0008][DS-F103]

可实现的 16 位方案仍用 2 片，但改为扩展深度：两片 $\text{DQ}[15:0]$ 共享 FSMC $\text{D}[15:0]$ ，两片 $\text{A}[19:0]$ 共享 FSMC $\text{A}[19:0]$ （对应 CPU 字节地址 $\text{A}[20:1]$ ）；由 FSMC $\text{A}20$ （对应 CPU $\text{A}21$ ）选择前、后 2MiB 芯片。结合 $\text{NE}1$ 及剩余高地址译码，分别产生片选：

片 0 有效: $0\text{x}60000000 \leq A \leq 0\text{x}601FFFFF$,

片 1 有效: $0\text{x}60200000 \leq A \leq 0\text{x}603FFFFF$.

例如在 $\text{NE}1$ 的 64MiB 窗口内，再要求 CPU $\text{A}[25:22]=0$ ，用 $\text{A}21$ 二选一；未完全译码会产生地址镜像。 $\text{NBL}0$ / $\text{NBL}1$ 接两片的字节使能， NOE / NWE 接 OE / WE 。不能简单用 $\text{NE}1$ 、 $\text{NE}2$ 各接一片而声称得到连续 4MiB，因为二者的 FSMC 窗口起址相隔 64MiB。

(3) 地址范围。从外部 RAM 区起点分配连续 4MiB：

$$0\text{x}60000000 + 0\text{x}00400000 - 1 = 0\text{x}603FFFFF.$$

所以可选范围为 $\boxed{0x60000000-0x603FFFFF}$ ，包含首尾共 2^{22} 个字节。题目没有规定必须从区域首址开始，其他正确译码的、合法区域内连续 4MiB 区间也可设计。

第 46 题 Cache 计算：Tag 19 位；含有效位 532B；行号 2、0

以下假定数据 Cache 初始为空、只有题给数组产生数据访问，忽略预取及其他访问；循环变量和和数按题面放在寄存器中。

(1) **Tag 位数**。主存 $256\text{MiB} = 2^{28}\text{B}$ ，故地址 28 位；64B 行需要 6 位块内偏移，8 个直接映射行需要 3 位索引：

$$\boxed{t = 28 - 6 - 3 = 19 \text{ 位}}, \quad \underbrace{\text{Tag}}_{19} \mid \underbrace{\text{Index}}_3 \mid \underbrace{\text{Offset}}_6.$$

(2) **总容量**。直接映射没有组内替换选择，但每行仍需一个有效位，表示 Tag 和数据是否有效。按题目仅排除一致性及替换算法控制位的通常口径：

$$C = 8(64 \times 8 + 19 + 1) = \boxed{4256 \text{ bit} = 532\text{B}}.$$

其中数据本体是 512B，19B 是 Tag，1B 是 8 个有效位。若课堂额外约定连有效位也一并不计，则为 $8(512 + 19) = 4248 \text{ bit}$ ，即 531B；应注明这一额外假设，不能在“含 Tag 等信息”的题面下无声漏掉有效位。无需为直接映射另加 LRU 位。

(3) **两元素的行号**。按行优先、每元素 4B：

$$A(i, j) = 20 + 4(64i + j), \quad \text{line}(A) = \lfloor A/64 \rfloor \bmod 8.$$

所以

$$A(0, 31) = 144, \quad \text{line} = 2 \bmod 8 = \boxed{2},$$

$$A(2, 1) = 536, \quad \text{line} = 8 \bmod 8 = \boxed{0}.$$

首地址 20D 是十进制 20，即 $0x14$ ，不是 20 个字。

(4) **程序执行时间**。在题目的访存模型下程序 1 更快。它连续读相邻整数，可重复使用刚装入的同一个 Cache 行。程序 2 内层地址每次跨 256B，即 4 个块，索引每次加 4 (mod 8)，64 行数组在两条 Cache 行之间反复冲突；每次换列也无法保住所需的旧块。直接映射仿真给出程序 2 为 4096 次缺失、0 次命中。实际总时间还受编译和存储器延迟影响，此处比较的是题设对应的缓存效应。

(5) **程序 1 命中率**。数组占 $4096 \times 4 = 16384\text{B}$ ，字节范围为 $[20, 16403]$ 。它触及的主存块为 0 到 256，共 257 块；顺序访问每块第一次缺失，剩余元素命中：

$$N_{\text{miss}} = \left\lfloor \frac{20 + 16384 - 1}{64} \right\rfloor - \left\lfloor \frac{20}{64} \right\rfloor + 1 = 257,$$

$$\boxed{h_1 = \frac{4096 - 257}{4096} = \frac{3839}{4096} \approx 93.7256\%}.$$

首块含 11 个元素，最后一块含 5 个，中间 255 块各含 16 个，因此命中数也可写成 $10 + 255 \times 15 + 4 = 3839$ 。这一步解释了为什么不能直接套用 $15/16$ 。若 Cache 初始非空，命中率还依赖其初始内容，题面缺省采用冷 Cache。

第 47 题 GPIO 与 USART：寄存器地址、位带映射与收发

(1) GPIOE 基地址。

$$\text{GPIOE_BASE} = 0x40000000 + 0x00020000 + 0x1000 = \boxed{0x40021000}.$$

(2) GPIOE 的 IDR 地址。IDR 偏移为十进制 16，即 $0x10$ ，故

$$\text{GPIOE_IDR_Addr} = \boxed{0x40021010}.$$

(3) 宏 $\text{PEin}(n)$ 。它把 IDR 中第 n 位映射为可作字访问的位带别名左值；读取此别名得到对应输入位的 0 或 1。`volatile` 防止编译器把外设值当作不变值缓存。该宏访问的是输入数据，不是 ODR。

(4) GPIOD 时钟。使能 AHB1 总线上的 GPIOD 外设时钟，使后续引脚配置和端口操作正常工作；这一步本身不等于已经使能 USART2 的 APB1 时钟。

(5) PD6 复用。将 PD6 的复用选择设为 USART2。结合前一行 PD5 的配置，PD5 用作 USART2_TX，PD6 用作 USART2_RX；随后 GPIO 初始化还将二者模式设为复用。^[RM0090]

(6) 真正读取的地址。外设位带公式为

$$A_{\text{alias}} = 0x42000000 + (A_{\text{reg}} - 0x40000000) \times 32 + n \times 4.$$

代入 $A_{\text{reg}} = 0x40021010$ 、 $n = 4$ 得

$$A_{\text{alias}} = \boxed{0x42420210}.$$

CPU 对这个别名作 32 位 volatile 读，硬件转为读取 $0x40021010$ 的 bit4；读到 1 表示 DIP0 不接地，读到 0 表示接地。地址不是直接对 IDR 加 4，也不是 IDR 本身。

(7) 串口状态位。TXE=1 表示发送数据寄存器空，可写下一帧的数据；它不等于最后一个停止位已经发完，后者应看 TC。RXNE=1 表示接收数据寄存器有未读数据，可读 DR 获取；按正确序列读取会清除相应接收状态。^[RM0090]

原代码实际行为与修正。原头文件为 F1 的 `stm32f10x.h`，却使用 F4 的 GPIO / AHB1 API，不能据此视为一份可直接编译的 F407 程序。更关键的是第⑦行检查 USART1 的 TXE，而随后写 USART2；USART1 是否空不能保证 USART2 可以接受新数据。正确头文件应使用 F4 工程，发送门控应与数据寄存器属于同一串口。

在保留题目已省略的 GPIOE 上拉输入、USART2 波特率及收发使能初始化，并确保 USART2 的 APB1 时钟已开启后，轮询主体可改为：

```

1  #include "stm32f4xx.h"
2
3  /* GPIOD_Config(); GPIOE input setup; USART2 setup;
4   RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART2, ENABLE);
5   USART_Cmd(USART2, ENABLE); must be done before this loop. */
6  for (;;) {
7      unsigned char value = (unsigned char)PEin(4);
8      if (USART_GetFlagStatus(USART2, USART_FLAG_TXE) == SET) {
9          USART_SendData(USART2, value);
10     }
11     if (USART_GetFlagStatus(USART2, USART_FLAG_RXNE) == SET) {
12         uc_rcv_ch = (unsigned char)USART_ReceiveData(USART2);
13     }
14 }
```

这样发送的是二进制字节 $0x00/0x01$ ，与题意的状态值一致。若希望终端显示字符 0 或 1，应发送 '0' + value；不能把原始字节 1 当成 ASCII 字符 '1'。题目没有要求收到数据后做其他处理。

第 48 题 EXTI 与低有效 LED：DIP1 触发；EXTI4 抢占

(1) 两条 BSRR 写操作。电路中 LED 阳极经电阻接 VCC，GPIO 拉低时灌电流，故低电平亮。

$$0x0C000000 = 2^{27} + 2^{26} \Rightarrow BR11 = BR10 = 1 \Rightarrow PG11 = PG10 = 0,$$

$$0x00000C00 = 2^{11} + 2^{10} \Rightarrow BS11 = BS10 = 1 \Rightarrow PG11 = PG10 = 1.$$

因此①两灯点亮，②两灯熄灭。BSRR 写 1 执行对应置位 / 复位，写 0 对其他引脚不作修改；高半部的“置 1”不是把输出引脚置高。^[RM0090]

(2) main 的实际功能。按可编译的语义看，它配置 PG10、PG11 为推挽输出，PE4、PE5 为上拉输入，只将 PE5 路由到下降沿 EXTI5，配置 EXTI9_5 的 NVIC 后忙等。故 DIP1 从高变低才触发 ISR，DIP0 并未配置为 EXTI4 中断源。ISR 根据 PG10 的输入电平同时开或同时关两灯，不是 DIP0、DIP1 各自独立控制一盏灯。

原代码没有明确设置两灯的初始 ODR。若从复位默认 ODR=0 开始，配置成输出后通常会点亮两灯；若在已有系统状态下再次初始化，不能单凭 GPIO_Init 推断输出初值。函数名 LED0_IsOn() 直接返回 PG10 电平，实际返回 1 代表 LED0 灭，名称与低有效逻辑相反。

(3) DIP0 决定 EXTI5 分支。保留原先“以 PG10 电平切换两灯”的规则；当 PE4=1 时，令 PG10=0、PG11=1。所需混合 BSRR 值为 $2^{26} + 2^{11} = 0x04000800$ ：

```

1  /* One 32-bit write to BSRR, independent of whether the
2     F4 header names it BSRR or splits it as BSRR1/BSRR0. */
3  #define WRITE_GPIO_BSRR(v) \
4     (*(volatile uint32_t*)(GPIO_BASE + 0x18u) = (uint32_t)(v))
5
6  void EXTI9_5_IRQHandler(void)
7  {
8     if (EXTI_GetITStatus(EXTI_Line5) != RESET) {
9         if (GPIO_ReadInputDataBit(GPIOE, GPIO_Pin_4) == Bit_RESET) {
10             if (LED0_IsOn()) { /* PG10 is high: currently off */
11                 WRITE_GPIO_BSRR(0x0C000000u); /* both on */
12             } else {
13                 WRITE_GPIO_BSRR(0x00000C00u); /* both off */
14             }
15         } else {
16             WRITE_GPIO_BSRR(0x04000800u); /* LED0 on, LED1 off */
17         }
18         EXTI_ClearITPendingBit(EXTI_Line5);
19     }
20 }

```

这里仅响应 EXTI5 事件并在事件发生时采样 DIP0，不会因 DIP0 单独变高而自动立即改变灯状态；这与本小问要求修改 EXTI9_5 处理函数的范围一致。

(4) 新增 EXTI4 及抢占。保留 PE4 上拉输入，增加 PE4 到 EXTI4 的路由、下降沿检测及中断使能；启用 NVIC 的 EXTI4 通道。原 EXTI9_5 抢占优先级为 0，已最高，必

须把它改为 1，再把 EXTI4 设为 0。题给 PriorityGroup_1 有 1 位抢占优先级，正好允许 0 和 1；只调子优先级不能满足打断要求。

```

1  /* Add inside EXTI_Config after SYSCFG clock is enabled. */
2  SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOE, GPIO_PinSource4);
3  EXTI_InitStructure.EXTI_Line = EXTI_Line4;
4  EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
5  EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
6  EXTI_InitStructure.EXTI_LineCmd = ENABLE;
7  EXTI_ClearITPendingBit(EXTI_Line4);
8  EXTI_Init(&EXTI_InitStructure);
9
10 /* In NVIC_Config, same priority grouping for both IRQs. */
11 NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
12 NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;
13 NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
14 NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
15 NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
16 NVIC_Init(&NVIC_InitStructure);
17 NVIC_InitStructure.NVIC_IRQChannel = EXTI4_IRQn;
18 NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
19 NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
20 NVIC_Init(&NVIC_InitStructure);
21
22 void EXTI4_IRQHandler(void)
23 {
24     if (EXTI_GetITStatus(EXTI_Line4) != RESET) {
25         WRITE_GPIOD_BSRR(0x00000C00u); /* both off */
26         EXTI_ClearITPendingBit(EXTI_Line4);
27     }
28 }

```

以上实现“在 EXTI4 服务时熄灭、可抢占 EXTI5”。若另要求 EXTI4 后两灯一直保持熄灭，还需增加共享状态 / 事件顺序设计：被抢占的 EXTI5 恢复后可能继续写灯，抢占优先级本身不保证最后写入者仍是 EXTI4。原题并未要求锁定熄灭状态。

(5) 节能修改。初始化完成、保持所需 EXTI 唤醒能力后，把空转改为浅睡眠等待中断：

```

1  for (;;) {
2      __WFI();
3  }

```

CPU 处理完中断后返回，再次等待；还可关闭不用的外设时钟。若选择 Stop 等深睡模式，须另外配置唤醒及恢复系统时钟，不能无条件把深睡当成同一个修改。

原附录①缺少分号，严格按原文会编译失败；上面已经补齐。另附录 2 给出的 GPIO 结构体把 BSRR 分成 BSRRL / BSRRH，直接写 GPIOD->BSRR 未必匹配该版本头文件；上面的 32 位地址宏与其偏移 0x18 一致，避免字段名差异。原卷 EXTI4 的表格说明误印为 Line0，不应据此把程序路由改成 EXTI0。

第 49 题 图示串行协议：2 字节；写指令依次发送 0xF8、0x01

(1) 帧长。严格按图 3 的 16 个时钟，前八位为五个 1、RW、RS、0，后八位为 D7-D0：

$$\underbrace{1\ 1\ 1\ 1\ 1\ RW\ RS\ 0}_{\text{同步 / 控制字节}}\ \underbrace{D7\ D6\ D5\ D4\ D3\ D2\ D1\ D0}_{\text{有效数据字节}}.$$

所以发送一个有效字节对应 16 bit，即 2 字节 的题示帧。

(2) 按 MSB 优先发送一个串行字节。每次先拉低时钟再设置 SID，低电平期间留出建立时间，再拉高并保持数据：

```

1  typedef unsigned char u8;
2
3  void SendByte(u8 byte)
4  {
5      u8 mask;
6      for (mask = 0x80u; mask != 0u; mask >>= 1) {
7          SCLK = 0;
8          SID = ((byte & mask) != 0u) ? 1 : 0;
9          delay_us(1); /* data setup / low phase */
10         SCLK = 1;
11         delay_us(1); /* data stable throughout high phase */
12     }
13     SCLK = 0;
14 }

```

mask 从 0x80 移至 0x01 后变 0，恰好发八位，不存在无符号计数器从 0 减到 255 的死循环。每个位的 SID 都在 SCLK=0 后才改变。示例延时值用于表达时序关系；题面未提供最小高低宽度，实际应用须取满足器件时序的值。

(3) 发送控制指令 0x01。MCU 向外设写入故 RW=0，数据为控制指令故 RS=0；头字节为二进制 11111000：

```

1  SendByte(0xF8u); /* 11111, RW=0, RS=0, 0 */
2  SendByte(0x01u); /* D7..D0: command 0x01 */

```

此处总计两次调用、16 个 SCLK 脉冲。图示没有要求在 D4 与 D3 之间插入补零字节，不应自动改成三次调用。SendByte 负责原始 8 位发送，帧头由调用者组织，不能让函数每次都自行重复插入帧头。

第二章 2024–2025 学年：答案及解析

答案速查

本章按核验后的原题作答。单位 kB、MB 按本卷习惯取 2^{10} B、 2^{20} B。组合选项按“所列各项均符合题干”判断；多个选项成立时明确列出，不凭空补充授课约定。

题号	填空答案（按空格次序）
1	4294967279; 255
2	-17
3	0xEF; -17
4	0x20000000--0x2000FFFF
5	0x08000000--0x08017FFF
6	预期入口 0x08001000; 原表向量 bit 0 错误，实际正常启动不成立
7	0x20000300--0x200003FF; 压栈数据值无法确定
8	0x20000008; 0x00FF2027
9	0x22000028; 0x00000000
10	默认对齐配置下 BusFault，未使能时升级 HardFault；配置非对齐陷阱时可为 UsageFault

题号	11	12	13	14	15	16	17	18	19	20
答案	D	C	C	E	B	F	D	D	C	A

题号	答案	题号	答案
21	F; 术语另一解释为 C	26	B、D（原题多解）
22	C	27	C
23	半同步命名下 E; 拒绝该命名则 C、D、E	28	E
24	常规存储布局下 B; 严格依实现	29	A
25	C	30	E

题号	31	32	33	34	35	36	37	38
答案	√	√*	×	√	×	√	√	√

题号	39	40	41	42	43	44	45
答案	×	×	√*	√	×	√	×

* 第 32 题限第一代 DDR；第 41 题限本卷 Cortex-M3/M4 语境。题 21、23、24、26 的条件和多解原因见逐题说明；第 27 题以标准入出栈操作解释“栈的访问”。

一、填空题

第 1 题 4294967279; 255

关键词：无符号数、饱和。32 位无符号解释为

$$0xFFFFFFFF = 2^{32} - 1 = 4294967295.$$

8 位无符号范围为 $[0, 255]$ ，因此饱和结果为 $\min(4294967279, 255) = 255$ 。截取低 8 位会得到 239，这是截断结果，不是本题规定的饱和结果。

第 2 题 -17

关键词：补码。最高位为 1，32 位补码的真值为 $4294967279 - 2^{32} = -17$ 。也可取反加一得到幅值 $0x11$ ，再加负号。

第 3 题 0xEF; -17

关键词：符号扩展。低 8 位保留原始机器数 $0xEF$ ；其真值为 $239 - 256 = -17$ 。最高位为 1，新增 24 位全补 1，即得到题设 $0xFFFFFFFF$ 。

第 4 题 0x20000000--0x2000FFFF

关键词：容量与地址范围。 $64\text{ kB} = 0x10000\text{ B}$ ，末地址为起始地址加容量减 1，即 $0x2000FFFF$ 。这是 SRAM 的直接存储地址范围；位带别名是同一存储位的另一种访问入口，不新增实体容量。

第 5 题 0x08000000--0x08017FFF

关键词：Flash 地址。 $96\text{ kB} = 0x18000\text{ B}$ ，故末地址为 $0x08000000 + 0x18000 - 1 = 0x08017FFF$ 。题目给出的容量是假设条件，不应擅自换成某一实际 F407 型号的容量。

第 6 题 预期入口 0x08001000; 原表不能保证正常启动

关键词：复位向量、Thumb 状态。复位先从地址 0 取初始 MSP，再从地址 4 取复位向量。若只问表中给出的程序入口地址，答案是 $0x08001000$ 。

原表的复位向量确为偶数 $0x08001000$ 。真实 Cortex-M4 将向量 bit 0 装入 T 位，该位必须为 1；这里会产生非法执行状态故障，通常升级为 HardFault，后续是否能进入故障处理程序还取决于未给出的故障向量等条件。因此不能断言它实际正常执行的第一条指令就在该地址。使预期启动成立的向量字应为 $0x08001001$ ，其实际指令地址仍为 $0x08001000$ 。^[ARM-M4]

第 7 题 0x20000300--0x200003FF; 数据值无法确定

关键词：满递减栈、信息不足。采用题表给出的初始 MSP 为栈顶，256 B 的预留栈区是

$$[0x20000400 - 0x100, 0x20000400 - 1].$$

首次压入一个 32 位字时，先减 SP 至 $0x200003FC$ ，再写该字。题目问的是“数据值”，没

有提供 PUSH 的源寄存器或异常现场，故无法确定；表中 `0x20000400` 原存储的字也不是首次压栈值。若发生异常一次压入多个字，栈帧总大小还须另算。^[ARM-M4]

第 8 题 `0x20000008; 0x00FF2027`

关键词：AHB 地址/数据相位。图中 B 属于先前地址 `0x20000008` 对应的读事务；不能把数据波形所在列当前的 HADDR 当成该数据的地址。LDR R1,[R0] 的有效地址就是 R0，因而执行前 R0 为 `0x20000008`，执行后 R1 为表中该地址处的字 `0x00FF2027`。^[AHB]

第 9 题 `0x22000028; 0x00000000`

关键词：等待周期、位带、小端。C 的数据相位被 HREADY 拉长，相应地延长了下一事务的地址相位；D 对应 `0x22000028`，不是后续 `0x0000003C`。位带反算为

$$0x28 = 40 = 32 \times 1 + 4 \times 2.$$

因此访问的是 SRAM 第 1 个偏移字节，即 `0x20000001` 的 bit 2。字 `0xFF002025` 的小端字节次序为 25 20 00 FF，第二个字节 `0x20` 的 bit 2 为 0；位带别名字读回整数 0，故 R1 为 `0x00000000`。^[ARM-M4]

第 10 题 默认情形 BusFault；可升级 HardFault

关键词：无效地址、非对齐故障。先有 `R4 = 0x2024FFFF`，随后 LDR 试图读取该地址起的字。它超出题设已实现 SRAM，也不在该系统的 Flash 中。普通 LDR 在复位默认 `CCR.UNALIGN_TRP=0` 下可进行非对齐访问；对题设未实现地址的读产生数据总线故障 BusFault。若 BusFault 未使能，故障升级为 HardFault。若另设 `UNALIGN_TRP=1`，则非对齐访问可先触发 UsageFault；MPU 等额外配置也会改变最先报告的故障。题目未给这些配置，不能把所有情形都写成同一种异常。^[ARM-M4]

二、单项选择题

第 11 题 D

关键词：IEEE 754 单精度。符号位 $s = 0$ ，阶码 $E = 142$ ，尾数字段 $F = 0x300000$ ，因此

$$(-1)^s(1 + F/2^{23})2^{E-127} = (1 + 3/8)2^{15} = 1.375 \times 2^{15} = 45056.$$

A、B、C 分别为 6144、12288、22528，只有 D 等于该值。

第 12 题 C

关键词：整型范围、浮点有效位。 α 落在 32 位有符号整数范围内； β 的整数值最大达 2^{40} ，32 位整数和只有 24 位有效二进制位的单精度无法完整精确表示该范围的所有整数。双精度有 53 位有效位，可以覆盖。

A 错： β 超过 32 位整数范围。B 错： β 丢失整数精度。C 满足精度要求，且 α 可直接采用处理器原生整数运算。D 也能满足数值精度，但将本可用整数处理的 α 改为浮点并无题

设支持的性能收益；按题目“最适宜”的通常原生整数运算假设选 C。具体速度仍依算法、处理器及编译器，不能把整型必然快于所有浮点操作当成普适定理。

第 13 题 C (I、VI)

关键词：进制换算。三个数为 8229, 8228, 2025。

I 真：2025H 最大。II 假：二进制数比它小 1。III 假：十进制 2025 最小。IV 假：首数最大。V 假：二进制数居中。VI 真：2025D 最小。

只有 C 的 I、VI 全真；A 含 V，B 含 II，D 含 II、V，E 含 III，均不满足。

第 14 题 E ($c + d$)

关键词：补码有符号溢出。真值为 $a = 2147483637$ 、 $b = 15$ 、 $c = -2$ 、 $d = -2147483647$ 。

选项	运算	数学真值	是否越出 32 位补码范围
A	$a - b$	2147483622	否
B	$a + c$	2147483635	否
C	$b + c$	13	否
D	$b + d$	-2147483632	否
E	$c + d$	-2147483649	是，低于 -2^{31}

E 的机器结果回绕为 0x7FFFFFFF；两个负数相加得到非负机器数， $V = 1$ 。这里须假设使用更新条件标志的算术指令，例如 ADDS；不更新 APSR 的 ADD 不保证改变 V。

第 15 题 B

关键词：存储层次、局部性。Cache 用较小的高速存储器保存常访问内容，主要缓解 CPU 与主存的速度差，故 B 正确。A、E 是容量问题；C 是主存与辅存间的速度差；D 把层次两端直接对应，均不是“Cache-主存”这一级的具体作用。

第 16 题 F (只有 II 错，A-E 均不合)

关键词：流水线冒险、停顿。

I 真：后一指令需要尚未生成的结果是典型 RAW 数据相关。

II 假：多个指令同时读取同一个寄存器可以合法进行；仅“同名寄存器”不充分构成数据冲突，还须看读写依赖或端口资源。

III 真：分支去向未确定会形成控制冒险。

IV 真：让依赖者停顿至结果可用，可避免数据冒险造成错误。

V 真：等待共享资源释放，可避免资源冲突。

VI 真：分支未决时暂停后续有效取指/执行，插入气泡直到分支确定，也能正确处理控制冒险。

VII 真：动态分支预测是降低控制冒险平均开销的常用方法。

每个 A-E 选项都至少带入一个真命题，因此选 F。若将“消除”严格限定为“删除语义依赖”，IV、V、VI 都需重述；不能只对 VI 使用这一限定，却把 IV、V 的停顿处理视为消除。无依据把“气泡不能处理控制冒险”判真。

第 17 题 D (II、III、V; V 按时钟周期理解)

关键词：吞吐率、三级流水线。

I 假：提升性能对应降低平均 CPI，而非提高 CPI。II 真：理想流水重叠提高 IPC。

III 真：Cortex-M3/M4 为取指、译码、执行三级。IV 假：所列五级不是该内核的三级划分。^[ARM-M4]

V 按统一流水时钟周期理解为真： $T_{clk} \geq \max(t_1, t_2, t_3) + t_{reg}$ ，由最慢一级及寄存器开销限制。VI、VII 假：取最小值或平均值不能保证最慢级及时完成。

对应 D。严格说，各级自身组合逻辑延迟不必相等，题目 V 的“每个阶段执行时间”应指统一分配的拍长；若指级内多个串行子操作，则应计其总延迟，而非简单取最大值。

第 18 题 D (IV、VI)

关键词：指令获取、总线、Thumb-2。

I 真：PC 与指令地址关联，读取 PC 时还须注意架构规定的偏移。II 真：CODE 区可存指令。III 真：可执行 SRAM 也可存指令。

IV 假：取指必须给出地址，再取得指令编码。V 真：Thumb-2 含 16 位、32 位编码。VI 假：执行访存等指令同样需要地址通路；并非整个过程只用数据总线。

D 的 IV、VI 全假；其余选项包含真项。^[ARM-M4]

第 19 题 C (II、IV)

关键词：相联度、冲突失效。

I 假：容量相同、条件可比时，提高相联度通常减少冲突，不是通常降低命中率。II 真：直接映射一般比组相联更容易冲突。III 假：组相联需多路标签比较、路选择与替换状态，硬件一般更复杂。

IV 真：组内路数常取较小值，如 2、4、8、16。V 假：组数由容量、行大小和相联度决定，不能一般断言很小。VI 假：F407 的 Flash ART 指令缓存为 64 行、每行 128 bit，手册说明满后使用 LRU 选择替换行；直接映射则每个主存块只能进入固定一行，不需要这种替换选择。不能仅凭 M4 内核本身没有通用 Cache 就忽略芯片的 ART。VII 假：通用 CPU 的各级 Cache 通常采用组相联。

C 全真，其他选项均夹带错误项。上述“一般”不是对任意替换策略和任意访问流的数学保证。^[RM0090]

第 20 题 A (I、II、III; V 也真)

关键词：存储介质。

I 真：片上 Flash 是半导体存储。II 真：SSD 的数据介质通常为 NAND Flash。III 真：U 盘通常也用 Flash。IV 假：HDD 的主要数据介质为磁盘；带控制器芯片不等于其主体采用半导体存储。V 真：PC 内存条通常为 DRAM。VI 不能作为普遍真命题：耐冲击、保持时间、擦写寿命、环境可靠性是不同指标，Flash 也有有限擦写寿命。

明确真项为 I、II、III、V；A 所列三项全部为真，而其他 A-E 选项都含 IV，故选 A。无需因 A 未穷尽 V 而选 F。

第 21 题 统一架构地址空间解释下 F；特定“同地址副本”解释下 C

关键词：地址空间、Cache 与 TCM。

I 真： n 根二进制地址线最多给出 2^n 种地址。II 真：存储器地址标识存储单元。

III、IV 均假：“必须共用”与“必须独立”都不是所有体系的要求；独立 I/O 编址也不等于必须有另一套物理地址引脚。

V 按软件可见地址解释为真：普通 Cache 保存某些主存地址的数据副本，软件不会因为命中而改用另一套 Cache 地址。

VI 的表述有歧义。TCM 是映射到处理器物理地址空间中的真实存储，通常与其他 RAM 占不同地址范围，但它们仍可属于同一个架构地址空间；“不同地址范围”不等于“不同地址空间”。按原句的架构地址空间含义，VI 真。若题目实际想表达“TCM 和其他主存是相同地址数据的自动镜像”，该解释为假。

VII 真：虚拟地址可经地址转换得到物理地址，未映射或无权限时转换可失败。

按第一种严格区分术语的读法，错误项仅 III、IV，A-E 无全假组合，选 F。按第二种、将 VI 理解为“透明副本”的读法，III、IV、VI 错，C 成立。不能把这两种命题无说明地混为一谈。[ARM-TCM]

第 22 题 C (IV、VII)

关键词：总线时序、仲裁。

I 真：共享公共传输通道是总线的基本思想。II 真：同步总线的接口传输遵循公共时钟，并不要求设备内部所有逻辑频率完全一致。III 真：传统异步握手可按互锁程度作三类划分。

IV 假：半同步时序仍有公共时钟，只增加等待握手；异步时序才不依赖统一传输时钟。

V 真：发起事务者为主设备、响应者为从设备。VI 真：单主环境基本周期可按寻址和数据传送分析。VII 假：不存在其他竞争主设备时，无需为分配总线主控权进行多主仲裁。

C 的 IV、VII 全假；其他选项含真项。

第 23 题 同步加等待称“半同步”时 E；拒绝该命名则 C、D、E

关键词：AHB 同步握手、突发拍数。

I 真：完整 AHB 体系可支持多主；AHB-Lite 是单主接口简化。II 假：HRDATA 与 HWDATA 为分离的读/写数据通路。III 真：传输时序以 HCLK 上升沿为基准。

IV 取决于“半同步”的命名约定：HREADY 引入整数个时钟等待，所有采样仍同步于 HCLK。若将“同步时钟加等待信号”这一机制称为半同步，IV 为真；这与 AMBA 规定同步采样并不矛盾。若题目要求严格只使用 AMBA 的同步协议术语而拒绝“半同步”命名，则 IV 不被接受。题面没有给出这一术语边界，不能仅凭同步采样就断言 IV 是明确的技术错误。

V 真：本事务数据相位可与下一事务地址相位重叠。VI、VII 均假：INCR4 指 4 个 beat，每拍的字节数由 HSIZE 决定，总字节数为 $4 \times 2^{\text{HSIZE}}$ ，不是无条件等于 4 B 或 4 个字。[AHB]

按“同步加等待称为半同步”的命名，IV 真，只有 E (II、VI、VII) 全假。若拒绝该命名、将 IV 判假，则 C (II、IV、VI)、D (II、IV、VII)、E 均满足；A、B 含 III 等真项而不满足。这是术语导致的条件答案，两种口径必须随答案一起给出。

第 24 题 常规存储布局下 B；仅据对齐条件无法唯一选出 B

关键词：字对齐、可写 RAM、实现差异。

对“不会是”逐项判断：I 在通常启动映射下为向量/代码区域，不作为普通临时变量位置，条件成立时判真；但 CODE 区的架构属性不是“永远只读”，可映射 RAM，不能仅凭地址 0 证明所有 Cortex-M4 系统都不可能使用它。

II 假：0x20252024 四字节对齐、在 SRAM 大区，只要有相应 RAM 即可。III 在默认存储图下为厂商系统/Device 区，通常不用于普通临时变量，判真同样依赖该区实现。IV 真：0x22002025 低两位为 01，不满足题目要求的字对齐，且属于位带别名而非普通 float 存储。

V、VI 假：0x20250004、0x20250008 都对齐，位于 SRAM 大区。

在“常规 CODE/系统区不放普通临时变量，给出的 SRAM 地址已实现”的假设下 I、III、IV 符合，选 B。题面只说“某 Cortex-M4 系统”，未指定映射、RAM 容量、MPU 属性；严谨地说，只有 IV 的非对齐能直接证明，其他地址的可用性须查具体系统，不能把 B 写成无条件硬件定律。^[ARM-M4]

第 25 题 C (I、III、V)

关键词：特权、模式、指令状态。

I 假：复位为特权线程模式。II 真：复位使用 MSP。III 假：Cortex-M4 不支持 AArch64，执行 M-profile 的 Thumb/Thumb-2 指令。IV 真：具有 Thread 和 Handler 两种模式。V 假：非特权代码受到系统区、特殊寄存器和可选 MPU 权限约束。VI 真：非特权级无法访问所有特殊寄存器。

C 全由假项组成，其他选项含 II、IV 或 VI 等真项。^[ARM-M4]

第 26 题 B、D (按原卷地址文字，多解)

关键词：存储图、端序、位带容量。

I 真（实现配置含义）：M3/M4 可由芯片实现选择大小端；不表示 STM32 用户可随意在运行中切换，F103/F407 的实际系统为小端。

II、III 均不成立：原文区间为 0x20000000--0x0001FFFF，上界小于下界，不能当作有效递增地址段，更不能默默把上界改成 0x2001FFFF。

IV 假：SRAM 可存放代码并取指。V 真：外部 RAM 区可以存放并执行代码。VI 假：1 MB × 8 bit/B × 4 B/bit = 32 MB，不是 16 MB。VII 真：访问须遵守存储类型、执行属性、权限等。^[ARM-M4]

错误项为 II、III、IV、VI。B (II、III、IV) 和 D (III、IV、VI) 都全错，故为多解。若将两处上界另行勘误为 0x2001FFFF，II 才变真，届时只有 D；该勘误不等于原卷答案。

第 27 题 C (标准入出栈语境；III、V、VII，IV 也错)

关键词：满递减双栈、权限。

I 真：有 MSP、PSP。II 在标准 PUSH/POP、异常入出栈语境下真：SP 至少四字节对齐，AAPCS 公共调用边界另有八字节对齐要求。若泛指对栈所在 RAM 的任意访问，栈上字节变量可用 LDRB/STRB [SP,#1] 访问，不能说该字节地址也必须四字节对齐。

III 假：非特权线程也能访问其有权限的栈。IV 假：线程是否使用 MSP/PSP 与是否特

权是不同控制维度，非特权线程可以使用 MSP；“不能用特权指令改 MSP 特殊寄存器”不等于“不能用 MSP 作当前 SP”。

V 假、VI 真：满递减栈压入新项后 SP 减小。VII 假：可将栈放在可写且满足访问属性的外部 RAM 等区域，非仅限 `0x20000000--0x3FFFFFFF` 大区。

按标准入出栈语境，C 的 III、V、VII 全假，其他 A-E 选项含 II 或 VI 真项，选 C。若把 II 泛化至任意栈内字节访问，则 II 也假，A（II、III、V）与 C 同时成立。IV 虽也是错项，选项不要求列出所有错项。^{[ARM-M4][AAPCS32]}

第 28 题 E（II、V）

关键词：STM32F103 时钟树。

I 真：SYSCLK 可用内部 HSI 或外部 HSE 路径。II 假：SYSCLK 的选择为 HSI、HSE、PLL 输出，不能直接选 LSE、LSI。III 真：通用/高级定时器可沿系统时钟树得到时钟，另有外部计数源不影响“可”字。

IV 真：SysTick 可选 HCLK 或 HCLK/8。V 假：独立看门狗由 LSI 驱动。VI 真：GPIO 的 APB2 时钟来自系统时钟树。

E 全假；其他选项包含 I、III 或 IV 真项。^[RM0008]

第 29 题 A（I、IV、VI）

关键词：存储器映射 I/O、串行设备地址。

I 真：STM32 的外设寄存器与内存统一编址。II 假：TIM 的 CR1/CR2 是 I/O 控制寄存器。III 假：USART 的 SR 是 I/O 状态寄存器。IV 真：GPIO 的 ODR 属于 I/O 寄存器。V 假：SPI 的 CR1/CR2 同样属于 I/O 寄存器。

VI 真：I²C 协议中的从机地址是总线设备标识，不是 CPU 地址空间中的寄存器地址。VII 假：器件内部单元地址通常要在从机寻址之后另传，其格式由器件协议规定，二者不同。

A 全真，其余选项包含假项。^{[RM0008][RM0090]}

第 30 题 E（III、V、VII）

关键词：异常向量、优先级、屏蔽。

I 真：异常向量表通常保存服务入口，首项 MSP 是已知特例。II 真：M3/M4 每项一个 32 位字。III 假：多个向量可指向同一个默认处理函数，保留项也可相同。

IV 真：PRIMASK 可屏蔽可配置优先级异常，Reset、NMI、HardFault 不受其屏蔽。V 假：子优先级本身不产生抢占权；抢占比较的是组/抢占优先级。VI 真：更高抢占优先级的异常在未被屏蔽等通常条件下可抢占低者。VII 假：地址 0 的首字用于初始 MSP，复位入口表项在地址 4。

E 全假，其他 A-E 选项均混有真项。优先级“高”指更高紧迫性，在 NVIC 编码中通常对应更小数值。^[ARM-M4]

三、判断题

第 31 题 ✓

关键词：指令与微操作。取指、寄存器读写、ALU 运算、存储器访问等是完成机器指令所需的基本动作；指令可在控制层分解为一系列微操作。并不要求每一条机器指令都用微程序控制实现。

第 32 题 ✓（第一代 DDR）

关键词：2n 预取。第一代 DDR SDRAM 采用 2n-prefetch，内部一次预取两个 n 位数据单元，再通过外部 n 位接口在两个边沿传输。这里“2 bit”是每个数据引脚对应的预取深度，不是整个芯片每次总共只读两位。若把 DDR 泛指 DDR2/DDR3/DDR4/DDR5，则该陈述不成立，各代预取深度并非都为 $2n$ 。

第 33 题 ✕

关键词：可擦除 ROM。EPROM 通常用紫外线擦除；EEPROM 和 Flash 才采用电擦除。可电编程不等于可电擦除。

第 34 题 ✓

关键词：高速串行接口。SATA、PCIe 5.0、USB4 都采用串行链路。PCIe 可以具有多条 lane，各 lane 内仍是串行传输，多 lane 不会把接口变成传统同步并行数据总线。

第 35 题 ✕

关键词：AHB 读写数据方向。从设备返回读数据使用 HRDATA，主设备采样它；HW-DATA 用于主设备向从设备提供写数据。^[AHB]

第 36 题 ✓

关键词：DMA 开销摊薄。配置好源、目的和长度后，DMA 自动搬运数据，不需 CPU 对每个数据项执行读写循环。大块传输更能摊薄配置和完成中断开销；具体速度仍取决于总线竞争和设备带宽。

第 37 题 ✓

关键词：写缓冲、可见性。写缓冲把写指令完成与外部写事务完成解耦，使处理器可继续执行后续指令；这本身不把程序变成乱序指令执行。其他部件观察到的存储访问顺序则还受存储类型、缓冲和屏障影响，需要时使用 DMB/DSB；不能把“顺序执行”误认为“每次外部写都已完成”。^[ARM-M4]

第 38 题 ✓

关键词：SP 低位约束。 M3 的栈指针低两位固定为 0。对 MSP 写入 0x40002025 不能保持题给精确数值，低两位被忽略后为 0x40002024。另外，该地址属于外设区域，能把某个对齐值写入 SP 不代表该处适合建栈。[ARM-M3]

第 39 题 ×

关键词：MMU、MPU、Cache。 本卷 M3/M4 不具备进行虚拟地址转换的 MMU；可选 MPU 管理访问权限和属性。Cache 的标签、替换及维护是缓存控制机制的职责，不能笼统说 Cortex-M 通过 MMU 管理 Cache。[ARM-M4]

第 40 题 ×

关键词：突发带宽。 突发传输连续发送多拍，能够摊薄控制和仲裁开销、提高有效带宽；地址相位仍可与数据相位重叠。“以牺牲性能换灵活性”不是 AHB 突发机制的基本作用。[AHB]

第 41 题 ✓（本卷 M3/M4 语境）

关键词：Harvard 内核接口。 M3/M4 的 CODE 区可通过 I-Code/D-Code 接口访问，SRAM 大区通常通过 System 接口，内核侧物理通路区分。外部互连和存储目标仍可能共享，并非任何时刻都绝无冲突。若把陈述泛指整个 Cortex-M 家族，含不同总线组织的 M0 等，则不能无条件判真。[ARM-M3][ARM-M4]

第 42 题 ✓

关键词：外设位带。 0x40001800 在外设位带的首 1 MB 内。其寄存器 bit n 对应的别名地址可写为

$$0x42000000 + 0x1800 \times 32 + 4n = 0x42030000 + 4n \quad (0 \leq n < 32).$$

对别名的一次访问替代软件的读-改-写序列；底层硬件仍完成原子位操作。对具特殊读写副作用的寄存器，仍须遵从器件对该位的访问规定。[ARM-M3]

第 43 题 ×

关键词：架构上限与芯片实现。 不能把核心可支持的最大异常/IRQ 空间直接当作 F103 每个型号的固定向量表长度。例如中容量 F103 的外部 IRQ 编号到 42，连同 16 个核心槽为 59 个位置；其他密度型号不同，不是统一 256 项。保留槽、启动文件是否人为补齐等也应与“已实现中断数”区分。[DS-F103][RM0008]

第 44 题 ✓

关键词：AAPCS 调用约定。 AAPCS32 规定参数、返回值、寄存器保存责任和栈使用约束。例如常用前四个核心参数寄存器为 R0-R3，公共接口处 SP 须八字节对齐。[AAPCS32]

第 45 题 ×

关键词：ODR 有效位。GPIO ODR 的有效输出位为低 16 位， $0xFFFF0000$ 的低 16 位全为 0，因此输出锁存值为全低。若引脚配置为普通推挽输出，表现为低电平；输入/复用模式下不能仅凭 ODR 断言引脚实际电平。全高的低 16 位值应为 $0x0000FFFF$ 。^{[RM0008][RM0090]}

四、设计及综合题（五题全部作答）

第 46 题 4 片； $0x60000000--0x600FFFFF$

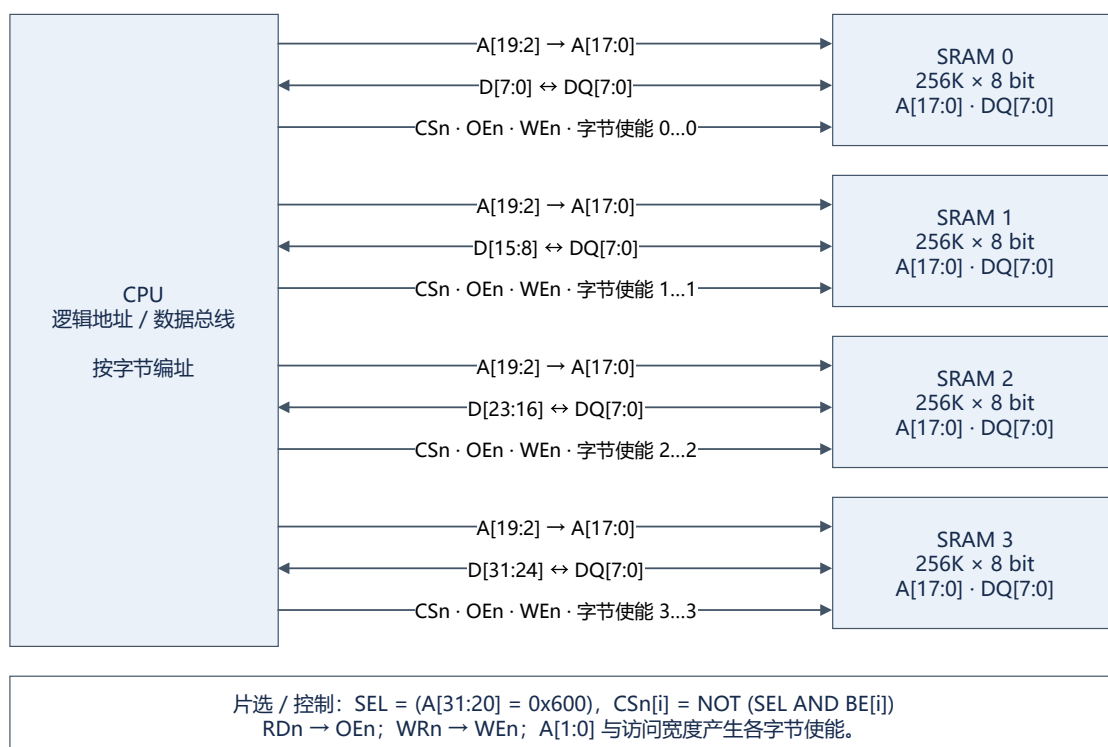
关键词：扩位、地址译码、字节选通。

（1）芯片数量。每片 $256K \times 8bit = 256KiB$ ，故 $1MiB/256KiB = 4$ 片。四片并行扩位形成 $256K \times 32bit$ 。

（2）连接。以下采用 32 位数据总线和小端字节通道约定。CPU 的字节地址低两位用于选择字节通道；四片芯片的片内地址是 CPU 字地址的低 18 位。

CPU/控制器信号	芯片连接
$A[19:2]$	并联至四片的 $A[17:0]$
$D[7:0]$	双向连接 SRAM0 的 $D[7:0]$
$D[15:8]$	双向连接 SRAM1 的 $D[7:0]$
$D[23:16]$	双向连接 SRAM2 的 $D[7:0]$
$D[31:24]$	双向连接 SRAM3 的 $D[7:0]$
$A[31:20]$	全译码： $SEL = 1$ 当且仅当该字段等于 $0x600$
访问宽度及 $A[1:0]$	产生 BE_0, BE_1, BE_2, BE_3 ，指示要访问的字节通道
片选、读、写	$\overline{CE_i} = \overline{SEL} \wedge \overline{BE_i}$ ，各片 $\overline{OE} = \overline{RD}$ ， $\overline{WE} = \overline{WR}$

2025 年·SRAM 位扩展连接（题设逻辑 32 位总线）



地址范围：0x60000000-0x600FFFFF；所有芯片共享上图所列地址信号。

图中 CSn/OEn/WEn 均低有效；各芯片仅在对应该字节通道被选中时使能。
按 32 位字访问时，4 个字节通道全部使能；按字节访问时仅使能目标通道。

整字对齐访问令四个 BE 同时有效；访问一个字节时只选中对应芯片，避免误写相邻字节。也可把通道选通接入独立 WE，保持读片选公用，但必须保留同样的字节写保护。控制器还须使读写时序满足 SRAM 要求，OE 与 WE 不可同时以冲突方式有效。

(3) 地址范围。高 12 位固定，低 20 位可变，共 2^{20} 个字节：

$$[0x60000000, 0x60000000 + 2^{20} - 1] = [0x60000000, 0x600FFFFF].$$

全译码高位可避免在其他地址形成未要求的镜像。

第 47 题 32 组、每组 4 行；两程序命中率均为 31/32

关键词：组相联、容量、访问顺序。

(1) Cache 组数与每组行数。

$$N_{line} = \frac{16 \times 1024}{128} = 128, \quad N_{set} = \frac{128}{4} = 32.$$

故 32 组，每组 4 行；偏移 7 位、组索引 5 位、标签 $20 - 7 - 5 = 8$ 位。

(2) 主存“页”数。原题未定义页大小，不能将一个唯一页数作为架构必然结论。若“页”按整份 Cache 容量 16 KiB 划分，则有 $1\text{MiB}/16\text{KiB} = 64$ 页，每页 $16\text{KiB}/128\text{B} = 128$ 行。若按一路容量 4 KiB 划分页（即一个组索引加行内偏移周期），则为 256 页、每页 32 行。无论使用哪种“页”的命名，主存总共有 $1\text{MiB}/128\text{B} = 8192$ 个块；它们的实际组映射相同。

(3) 数组能否全装入。 $64 \times 64 \times 4 = 16384$ B, 恰等于 Cache 容量。数组从地址 0 开始, 覆盖主存块 0-127; 块号对 32 取模, 各组恰映射 4 块, 因此在题设无其他数据竞争时全部可驻留。只比较总大小还不够, 这里也已检查逐组占用。

(4) $a[0][31]$ 的位置。地址为 $(0 \times 64 + 31)4 = 124 = 0x7C$, 四字节范围 $0x7C--0x7F$; 其主存块为 $0x00--0x7F$, 映射 Cache 第 0 组任一路, 行内偏移 $0x7C$ 。组相联未规定首次分配哪一路, 也没有 CPU 可直接寻址的统一“Cache 地址”。若约定每一路的 4 KiB 数据区连续排列, 四个候选整行内部偏移范围为

$$0x0000--0x007F, \quad 0x1000--0x107F, \\ 0x2000--0x207F, \quad 0x3000--0x307F.$$

对应元素位于每个候选范围的最后四字节, 如第 0 路 $0x007C--0x007F$ 。这种内部编号只是明确声明的表示法, 不是原题已给定的物理地址。

(5) 两程序命中率。一行含 $128/4 = 32$ 个元素。程序 1 按行遍历, 4096 次访问中每个块第一次失效, 恰 128 次:

$$H_1 = \frac{4096 - 128}{4096} = \frac{31}{32} = 96.875\%.$$

程序 2 按列遍历: 第 0 列访问 64 个偶数块, 分布在 16 个偶数组, 每组恰 4 块, 故第 0 列 64 次失效, 第 1-31 列命中; 第 32 列引入另外 64 个奇数块, 分布在奇数组, 第 33-63 列命中。无块超过组容量, 故

$$H_2 = \frac{4096 - 64 - 64}{4096} = \frac{31}{32} = 96.875\%.$$

已用逐地址的 32 组、4 路 Cache 模拟复算, 两序列均为 3968 次命中、128 次失效。题设把 i 、 j 、 sum 放在寄存器中, 计算只统计数组读取; 并假定数组内容已有效初始化, 结果求和不触发 C 有符号溢出的未定义行为。

第 48 题 定时周期 1 s; 原代码不能保证控制图中 LED

关键词: 定时器分频、寄存器偏移、低有效 LED。

(1) `TIM2_Delay()`。按题干 8 MHz、各级分频/倍频均为 1:

$$f_{cnt} = \frac{8 \times 10^6}{7999 + 1} = 1000 \text{ Hz}, \quad T = \frac{999 + 1}{1000} = 1 \text{ s}.$$

函数使能 TIM2 时钟, 配置向上计数, 清更新标志, 启动后轮询至下次更新, 意图提供约 1 s 阻塞延时。原函数返回时没有停表; 配置结构体也未完整初始化。稳健实现应先默认初始化结构体、停止/清计数器、更新预分频、清标志, 结束后按用途停表。图 2 晶体标 16 MHz, 与题干 8 MHz 不一致; 本问按文字条件计算, 若实际采用 16 MHz 且其他系数相同则为 0.5 s。

(2) `main()`。先必须区分原代码与修正后的逻辑。原注释配置 PA3 输出、首句写 GPIOA, 但图中 LED 在 PB3, 循环又写 GPIOB; 附录还把 GPIOB 基址定义错。因此按原文编译运行, 不能保证 LED 按设想受控。若将 GPIOB 地址和 PB3 初始化修正, 图中 LED 为低电平点亮。PA2 接地时, 先将 PB3 置 1 (灭), 等约 1 s, 然后置 0 (亮); 接地持续时下一轮马上又置 1, 因此仅有很短的亮脉冲, 不是亮 1 s、灭 1 s 的均匀闪烁。PA2 不接地时每轮置 0, LED 持续亮。

(3) ①的起始地址。GPIOA 基址为 `0x40010800`；结构体 CRL、CRH、IDR 各 4 B，IDR 偏移 8，故读取起址为 `0x40010808`。虽然附件展示的是整端口读函数，位读函数同样读取 IDR 后取 PA2。

(4) ②的地址范围。附录函数判断 `BitVal != Bit_RESET`，故 `0x10` 走 BSRR 分支，写入的引脚掩码为 `0x00000008`，不是把参数 `0x10` 写入端口。严格按题给宏：

`GPIOB_BASE = 0x40000000 + 0x20000 + 0x400 = 0x40020400.`

BSRR 偏移为 `0x10`，32 位存储访问范围为 `0x40020410--0x40020413`。真实 F103 GPIOB 在 APB2，基址应为 `0x40010C00`，其 BSRR 是 `0x40010C10--0x40010C13`。附录错误宏可能把写操作送至别的外设或未实现地址。

(5) ③的功能。分号就是空循环体；更新标志为 RESET 时持续轮询，为 SET 时退出。不产生中断延时，也不自动停止计数器。^[RM0008]

可实现该逻辑的修正版片段。使用真实 F103 头文件，GPIOA 时钟、PA2 上拉输入与 GPIOB 时钟、PB3 推挽输出均先正确配置；PB3 原为 JTAG 引脚，须通过 AFIO 关闭 JTAG、保留 SWD。下列寄存器写法完整明确了定时器初始条件：

```
1 static void TIM2_Delay_1s(void)
2 {
3     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
4     TIM_DeInit(TIM2);           /* reset previous timer modes */
5     TIM2->PSC = 7999U;
6     TIM2->ARR = 999U;
7     TIM2->EGR = TIM_EGR_UG;     /* load PSC; reset count phase */
8     TIM2->SR = 0U;
9     TIM2->CR1 = TIM_CR1_CEN;
10    while ((TIM2->SR & TIM_SR_UIF) == 0U) { }
11    TIM2->CR1 = 0U;
12    TIM2->SR = 0U;
13 }
14 /* Before this: initialize PA2 pull-up and PB3 output;
15    GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE); */
16 GPIO_ResetBits(GPIOB, GPIO_Pin_3);
17 for (;;) {
18     if (GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2) == Bit_RESET) {
19         GPIO_SetBits(GPIOB, GPIO_Pin_3);
20         TIM2_Delay_1s();
21     }
22     GPIO_ResetBits(GPIOB, GPIO_Pin_3);
23 }
```

这保留题给“接地时先灭约 1 s”的逻辑；若另需均匀闪烁，必须在置低之后再加入一次延时，那属于改变原逻辑。

第 49 题 ③ EXTI_Line2; ④ EXTI2_IRQn; IMR = 0x00000004

关键词：EXTI 路由、NVIC、边沿触发。

(1) 宏定义。① `LED0(x)` 写 PB3，零参数使 `PB3=0`，非零使 `PB3=1`；低有效 LED 因而分别亮、灭。②读取 PA2 的 IDR 位，接地为 0，配置上拉且断开为 1。

(2) 补空及必要修正。配合 PA2 和 EXTI2_IRQHandler, ③为 EXTI_Line2, ④为 EXTI2_IRQn (外部 IRQ 编号 8, 不是向量槽号 24)。但只补两个空不能保证原代码正确: ③实际上位于 EXTI 配置函数; F103 应使用 AFIO API; 原行又把 GPIOG 的第 4 脚映射至 EXTI4, 与 DIP0 完全不符。应一并改为:

```
1 RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);
2 GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource2);
3 EXTI_InitStructure.EXTI_Line = EXTI_Line2;          /* 3 */
4 NVIC_InitStructure.NVIC_IRQChannel = EXTI2_IRQn; /* 4 */
```

(3) IMR。复位全 0, 仅使能 EXTI2, 则 $1 \ll 2 = 0x00000004$ 。该值是正确配置意图下的寄存器结果; 无法编译的原 F103/SYSCFG 混用程序没有可报告的实际运行后寄存器值。

(4) ISR 功能。先检查 EXTI2 待决; PA2 为 1 则写 BSRR 的 BS3 (bit 3), PB3 置 1、灯灭; PA2 为 0 则写 BR3 (bit 19), PB3 清 0、灯亮; 最后对 EXTI2 待决位写 1 清除。

(5) 整体变化。路由和 GPIO 修正后, 启动 LED0(0) 已使灯亮; 拨至接地的下降沿, 稳定 PA2=0, ISR 再次使灯亮; 拨回断开的上升沿未使能中断, 灯不变。理想无抖动情形下灯始终亮, 并非每拨一次翻转, 也不是开关松开就自动灭。若采样发生在开关回弹期间, 可能读到 1, 题目没有消抖机制, 不能据理想波形保证抗抖效果。

修正后的初始化/处理示例。以下与题给单边沿、采样控制语义一致; 调用前使能 GPIOA/B 并配置 PA2 上拉、PB3 推挽, 解除 PB3 的 JTAG 复用:

```
1 static void EXTI2_Config_Corrected(void)
2 {
3     EXTI_InitTypeDef e;
4     NVIC_InitTypeDef n;
5     RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);
6     GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE);
7     GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource2);
8     EXTI_StructInit(&e);
9     e.EXTI_Line = EXTI_Line2;
10    e.EXTI_Mode = EXTI_Mode_Interrupt;
11    e.EXTI_Trigger = EXTI_Trigger_Falling;
12    e.EXTI_LineCmd = ENABLE;
13    EXTI_ClearITPendingBit(EXTI_Line2);
14    EXTI_Init(&e);
15    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
16    n.NVIC_IRQChannel = EXTI2_IRQn;
17    n.NVIC_IRQChannelPreemptionPriority = 0;
18    n.NVIC_IRQChannelSubPriority = 1;
19    n.NVIC_IRQChannelCmd = ENABLE;
20    NVIC_Init(&n);
21 }
22 void EXTI2_IRQHandler(void)
23 {
24     if (EXTI_GetITStatus(EXTI_Line2) != RESET) {
25         GPIOB->BSRR = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_2)
26             ? 0x00000008U : 0x00080000U;
27         EXTI_ClearITPendingBit(EXTI_Line2);
28     }
29 }
```

若要求“每次下降沿翻转 LED”，须另写取反逻辑；若要求“跟随开关亮灭”，须配置双边沿。两者均不是原 ISR 当前实现。^[RM0008]

第 50 题 配置线速率 115200 bit/s；双开关接地时意图发送 0x00

关键词：UART 标志、帧开销、低有效输出。

(1) **TC**。Transmission Complete 表示一帧包括停止位均已发完，且发送数据寄存器中没有待发数据；可用于等候整帧发送结束。题中①执行清 TC 标志。

(2) **输入编码**。去除数字中非法空格后，`GPIOC->IDR & 0x00000003U` 保留 PC1、PC0，编码为 DIP1:DIP0，其值为 0、1、2 或 3；向 8 位变量赋值不会丢失这两位。

(3) **TXE**。Transmit data register Empty，表示可向发送数据寄存器再写入一个数据；移位寄存器仍可能在发送上一帧，故 TXE 不等于 TC。

(4) **RXNE**。Receive data register Not Empty，表示接收数据寄存器有尚未取走的数据，读取 DR 取出数据并清该状态；读取错误的 USART 实例无助于接收当前线路数据。

(5) **传输速率**。本配置波特率为 115200，二进制 UART 的线路位速率为 115200 bit/s。一帧含起始位 1、数据位 8、停止位 2、无校验，共 11 位，连续无间隙发送时每方向净载荷上限为

$$R_{byte} = \frac{115200}{11} = 10472.727 \dots \text{ B/s}, \quad R_{data} = \frac{115200 \times 8}{11} = 83781.818 \dots \text{ bit/s}.$$

若“最高”指芯片可重新配置达到的硬件极限，须另给 USART 外设时钟和过采样模式，不能从当前 115200 配置推成芯片硬件上限。

(6) **两开关都接地**。两输入位都为 0，修正后的程序发送二进制字节 0x00（不是 ASCII 字符 '0'）。PC 判为偶数，返回 0xFF。

(7) **补⑤**。PB0 对应 LED0、PB1 对应 LED1，均为低有效。一次 BSRR 写即可同时置复位两个引脚：

```
1 if (uc_recv_ch == 0xFFU) {
2     GPIOB->BSRR = 0x00010002U; /* PB0=0, PB1=1 */
3 } else {
4     GPIOB->BSRR = 0x00020001U; /* PB0=1, PB1=0 */
5 }
```

第一常量的 bit 16 清 PB0、bit 1 置 PB1；第二常量的 bit 17 清 PB1、bit 0 置 PB0。

原程序不能直接实现上述通信：0x0000 0003 不是合法 C 整数字面量；`ui_dip0_value`；未在所示程序中声明且无有效作用；初始化和 TXE 检查用 USART1，发送、接收和 RXNE 检查用 USART2。图上 PA9/PA10 对应 USART1，必须统一到 USART1。图 3 还把桥片 TxD 接 MCU TxD、RxD 接 MCU RxD；若标签按各器件自身方向定义，连接也错，应将桥片 TX 接 PA10/RX、桥片 RX 接 PA9/TX，并共地。因此按原文不存在已证明的“PC 实际收到 0x00”结果。

可运行逻辑的修正版。下面保留题设 UART 配置为 USART1、115200、8 数据位、2 停止位、无校验；GPIO 初始化须将 PA9/10 设为 AF7，PC0/1 上拉输入，PB0/1 推挽输出，接线按上述交叉方式。采用一发一收，避免无界发送掩盖接收处理：

```
1 /* GPIO and USART1 initialized as described above. */
2 for (;;) {
3     uint8_t tx = (uint8_t)(GPIOC->IDR & 0x00000003U);
```

```
4   while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET)
5       { }
6   USART_SendData(USART1, tx);
7   while (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET)
8       { }
9   uint8_t rx = (uint8_t)USART_ReceiveData(USART1);
10  GPIOB->BSRR = (rx == 0xFFU) ? 0x00010002U : 0x00020001U;
11 }
```

这段阻塞示例假定 PC 必定应答；实际应用可再增加超时和 ORE 等错误处理。PC 与 MCU 必须使用相同帧格式。^[RM0090]

第三章 2026 年试卷答案及解析

答案速查

本卷共 49 题。速查中的“条件”与正文约定一致；原题条件缺失时不人为补出唯一答案。

题号	答案
1	4294967288; 65535。
2	-8。
3	0xFFF8; -8。
4	0x20000000 至 0x2000BFFF。
5	0x08000000 至 0x0800FFFF。
6	名义入口 0x08000100; 原复位向量 Thumb 位错误, 严格说不能据此正常启动。
7	0x20000F00 至 0x20000FFF; 首次压入的值无法确定。
8	R0=0x20000004; R1=0xFFFF2026。
9	R0=0x22000048; R1=0x00000000。
10	通常为总线故障; 未使能 BusFault 时升级为 HardFault; 另见未对齐条件。
11-18	11 A; 12 C; 13 E; 14 A; 15 F; 16 A; 17 F; 18 C。
19	统一架构地址空间、V 指理论虚拟寻址上限时 A、C、D; 若 V 未确定则 A 可确认。
20	F (统一架构地址空间); 若把 TCM 误称的“同地址空间”另解释为同地址透明副本, 则 C。
21-23	21 C; 22 A (教材“半同步”口径; 严格协议术语另见正文); 23 E。
24	常规 Flash 启动/RAM 变量假设下 B; 仅按一般 Cortex-M4 架构不能无条件排除地址 0。
25-27	25 C; 26 按原文无效区间判错为 B、E, 明确勘误后 E; 27 字宽入出栈口径 D, 任意栈区访问口径 F。
28-30	28 C; 29 D; 30 E (按 USART 硬件功能, 另见上层协议语义)。
31-35	×, ×, √, √, √。
36-40	√, ×, √, √, √ (第 40 题按通常教材模型)。
41-45	×, √, ×, ×, ×; 第 43 题的教材阶段概括见正文。

一、填空题

第 1 题 4294967288; 65535

主题：无符号表示与饱和。 $0xFFFFFFFF8 = 2^{32} - 8 = 4294967288$ 。16 位无符号整数的范围为 $[0, 2^{16} - 1]$ ，饱和转换取上界 65535，不是截断低 16 位得到 65528。

第 2 题 -8

主题：补码真值。最高位为 1，按 32 位补码解释应减去 2^{32} ，故 $(2^{32} - 8) - 2^{32} = -8$ 。

第 3 题 0xFFFF8; -8

主题：符号扩展。保留低 16 位得 $0xFFFF8$ ，其符号位为 1，扩展时高 16 位全填 1；真值 $65528 - 65536 = -8$ ，扩展不改变真值。

第 4 题 0x20000000 至 0x2000BFFF

主题：容量与地址上界。 $48 \text{ KiB} = 48 \times 1024 = 0xC000 \text{ B}$ ，末地址为首地址加容量减 1，即 $0x20000000 + 0xC000 - 1 = 0x2000BFFF$ 。位带别名不增加实际 SRAM 容量。

第 5 题 0x08000000 至 0x0800FFFF

主题：Flash 地址范围。 $64 \text{ KiB} = 0x10000 \text{ B}$ ，因此最后一个有效字节为 $0x08000000 + 0x10000 - 1 = 0x0800FFFF$ 。

第 6 题 名义入口 0x08000100，原向量无效

主题：复位向量与 Thumb 状态。地址 0 的首字初始化 MSP，地址 4 的字给出复位处理程序入口及 Thumb 状态。按题目意图，处理程序地址为 $0x08000100$ 。

真实 Cortex-M 的异常向量最低位必须为 1；该位用于设置 Thumb 状态，不属于实际指令地址。原表写 $0x08000100$ ，最低位为 0，会导致无效状态故障，不能保证正常执行此处第一条指令。若要正常从该地址执行，向量表项应写 $0x08000101$ 。后续题目按各自给定快照分析，不能把这处原题错误传播成所有后题均无法作答。[ARM-M4]

第 7 题 栈区 0x2000F00--0x2000FFF；首次压入值未知

主题：满递减栈与信息充分性。初始 MSP 由首向量读得 $0x20001000$ 。预留 256 B 的满递减栈区为

$$[\text{MSP}_0 - 256, \text{MSP}_0 - 1] = [0x2000F00, 0x2000FFF].$$

若首次仅压入一个字，则先减 SP 再存数，写地址为 $0x2000FFC$ ，之后 SP 也是该地址。但是压入的值取决于 PUSH 的寄存器列表及其内容，或异常自动入栈时的寄存器现场；题面均未给出。表中 $0x20001000$ 原有的 $0x12342023$ 不是首次压栈值，且该地址在上述预留区之外。[ARM-M4]

第 8 题 R0=0x20000004; R1=0xFFFF2026

主题：AHB 地址/数据两阶段。 AHB 的本次地址阶段与前一次数据阶段重叠，不能将上下波形同一垂线上的地址和数据直接配对。图中 0x20000004 的地址阶段之后完成的数据是 B；随后 0x20000008 的数据阶段因 HREADY 等待而延长，对应 C。因此 LDR R1,[R0] 的 R0 为 0x20000004，由表 1 读得 R1 为 0xFFFF2026。^[AHB]

第 9 题 R0=0x22000048; R1=0

主题：AHB 配对与位带反算。 数据 D 对应先前被接受的地址 0x22000048；与 D 同时出现的 0x0000003C 属于下一笔地址阶段。位带别名公式为

$$A_{\text{alias}} = 0x22000000 + 32\Delta A + 4b.$$

差值 $0x48 = 72 = 32 \times 2 + 4 \times 2$ ，所以目标是 0x20000002 的 bit 2，也即从 0x20000000 开始的字的 bit 18。STM32 小端存储 0xFF002025 时，各字节依次为 25, 20, 00, FF；目标字节为 0，读别名得到整个字 0x00000000。^{[AHB][ARM-M4][RM0090]}

第 10 题 访问未实现存储器，通常为 BusFault 或升级的 HardFault

主题：有效地址与故障类型。 R4 读得 0x00FF2027，随后 LDR 将其作为地址，而不是继续访问原来的 0x20000008。题设仅实现指定 SRAM 和 Flash，0x00FF2027 不在有效存储器范围内；若总线对该非法访问返回错误，形成精确数据总线故障。BusFault 未使能时升级为 HardFault。该地址还非字对齐。普通 LDR 在普通存储器区域可支持未对齐访问；若设置 CCR.UNALIGN_TRP=1，也可能首先报告 UsageFault。因此没有给出故障使能、未对齐陷阱与总线响应细节时，应写清“无效存储器访问导致故障”，不能承诺唯一的最终异常入口。^{[ARM-M4][RM0090]}

二、单项选择题**第 11 题 A**

主题：IEEE 754 单精度编码。 $2026 = 0x7EA = 1111101010_2 = 1.111101010_2 \times 2^{10}$ 。符号 $s = 0$ ，阶码 $E = 10 + 127 = 137 = 10001001_2$ ，23 位尾数字段为 11110101000000000000000₂。拼接得到 0x44FD4000，即 A；其余选项的符号或阶码/尾数不符。

第 12 题 C

主题：不同进制的无符号数比较。 三数分别为 $2026H = 8230$ ， $0010\ 0000\ 0010\ 0100_2 = 2024H = 8228$ ， $2026_D = 2026$ ，故 $8230 > 8228 > 2026$ 。I 真（首数最大）；II 假；III 假；IV 假；V 假；VI 真（十进制数最小）。选项 A 含假的 V，B 含假的 II，C 的 I、VI 全真，D 含 II/V，E 含 III，故选 C，不选 F。

第 13 题 E

主题：有符号溢出。 $a = 2^{31} - 1$, $b = 15$, $c = -16$, $d = -2^{31} + 1$ 。A 的 $a - b = 2^{31} - 16$ ；B 的 $a + c = 2^{31} - 17$ ；C 的 $b + c = -1$ ；D 的 $b + d = -2^{31} + 16$ ，均在范围内。E 的 $c + d = -2^{31} - 15$ 小于最小值，硬件 32 位结果回绕为 `0x7FFFFFFF1`，同号负数相加得到正号，故 $V = 1$ 。这里讨论会更新标志的机器运算；若直接写 C 有符号溢出表达式，其语言语义是未定义行为，不能据此保证编译后 PSR 值。^[ARM-M4]

第 14 题 A

主题：Cache 的目的。 Cache 用较小而较快的存储器缓存近期/邻近访问的数据，缓解 CPU 与主存的速度差异，故 A。B、C 属于辅助存储层次问题，D、E 属于容量问题，不是 Cache-主存结构的主要目的；F 不成立。

第 15 题 F

主题：流水线冒险与停顿。 I 真，是典型 RAW 数据依赖；II 假，多个指令同时读取同一寄存器未必有数据冒险，争用同一端口还可能是结构冒险；III 真，分支会改变后续取指；IV 真，等分支确定后再推进可避免错误执行；V 真，停顿至生产者给出结果可处理数据冒险；VI 真，串行化资源使用可处理结构冒险；VII 真，动态预测常用于降低控制冒险代价，但不能保证永不误预测。题目问“不正确”，只有 II。A 含 I/IV 真项，B 含 IV 真项，C 含 VI 真项，D 含 III/IV 真项，E 含 IV/VI 真项，A-E 均不是全由错误陈述构成，故 F。插入气泡会降低性能，但“代价较大”不等于“不能解决”。

第 16 题 A

主题：三级流水线与吞吐率。 I 真，流水化可提高吞吐率 IPC；II 假，相应目标是降低 CPI；III 真，M3/M4 常用取指、译码、执行三级描述；IV 假，五阶段不是这两个内核的该模型；V 在教材同步流水线模型下真，共用时钟周期至少覆盖最慢级延迟（实际还要加寄存器开销）；VI、VII 假，最小值/平均值可能使慢级来不及完成。A 的 I、III、V 全真；B/C 含 VI，D/E 含 II，因此选 A。V 指各级共用的节拍，不是说每一级组合电路的实际传播延迟都相等。^{[ARM-M3][ARM-M4]}

第 17 题 F（按常见直接存储接口）

主题：MCU 辅助存储接口。 I SCSI、II IDE、III SATA 通常不是基础 MCU 的常见直接存储接口；IV I²C 真，常接 EEPROM；V SPI 真，常接串行 Flash/存储卡；VI UART、VII USART 通常用于串行通信，不是这里常见 EEPROM/Flash 的直接存储接口；VIII PCIe 也不属本题 MCU 场景。A/B 均包含不合项，C 含 SATA，D 的 IV、VI 含 UART，E 的 V、VI 也含 UART，因此按上述题意选 F。原卷 D 明确印作“IV、VI”，不能偷偷按“IV、V”作答。若额外引入 UART 控制的存储模块，VI 也可能成立，届时 D、E 可同时成立；这属于扩展设备场景，不能作为原题唯一标准答案。

第 18 题 C

主题：Cache 映射组织。 I 假，同容量、同块长等条件下全相联通常减少冲突未命中，不是命中率最低；II 真，全相联需比较更多标签，硬件复杂；III 真，直接映射更容易发生映射冲突；IV 真，组内路数通常较小，16 路是合理例子；V 假，组数由容量、块长和路数决定，不能笼统说通常只有 16 组；VI 假，不能把所有 STM32 的缓存一律称为直接映射，不少型号无通用 Cache；VII 假，通用 CPU 常用组相联。A/B 含 I，D 含 V，E 含 VII，C 的 II、III、IV 全真，故 C。关于具体 STM32，应查对应型号的 Flash 加速器/Cache 结构，而不是从“单片机”推断映射方式。[RM0090]

第 19 题 V 指理论虚拟寻址上限时 A、C、D；V 未确定时 A 可确认

主题：存储层次与地址空间。 I 真，通常将内存称为主存；II 假，外存块地址与 CPU 主存地址不是自动统一的地址空间；III 真，通常透明 Cache 使用同一程序访问地址，命中其副本不需要软件换用另一地址空间。IV 在统一架构地址空间的含义下真，TCM 通常作为处理器存储映射中的独立区域，与普通 RAM 共用处理器地址空间，并非二者占据相同地址。V 在“CPU 的虚拟地址位数确定理论最大虚拟空间”的语境下真；实际操作系统可用范围还可能受限，物理地址宽度也不能直接决定虚拟上限。VI 假，地址/数据总线可按时分复用。在上述 III、IV、V 口径下，真项为 I、III、IV、V；A、C、D 均由真项组成；B 含 II，E 含 VI。题目未要求选项穷尽所有真项，不能只认 A。若题目的“CPU 可寻址”未说明指虚拟还是物理范围，V 无法直接确认，此时 A 是不依赖 V 的可确认选项。Cache 内部组/路/偏移的编码与主存地址不同，并不证明两者拥有不同的程序可见地址空间；这与本书其他年份的 Cache 地址题保持同一概念。[ARM-M3][ARM-TCM]

第 20 题 F（统一架构地址空间）；TCM 另解为透明副本时 C

主题：统一编址、独立编址与透明缓存。 I 真，地址线宽影响可表示的地址数量；II 真，地址标识存储单元；III 假、IV 假，I/O 接口组织不能被概括为必须共用或必须独立物理地址线，独立编址也不等于必有两套地址线。V 真，透明 Cache 与主存使用同一程序访问地址空间，其内部查找编码不同不改变这一点。VI 按统一架构地址空间为真，TCM 可映射在处理器存储图中的独立地址范围；VII 真，虚拟地址经映射可得到物理地址。因此错误项只有 III、IV，而 A–E 每项至少含一条真陈述，应选 F。若某种教材用 VI 的“相同地址空间”不严谨地指“与主存相同地址内容的自动透明副本”，则这一另一个命题为假，III、IV、VI 同假时 C 成立；必须把该附加解释写明。不能以 Cache 内部有索引地址为由，把 V 静默判假而选 D。这与 2025 年问题的术语限定一致。[ARM-TCM]

第 21 题 C

主题：总线时序与主从关系。 I 真，共享通道是总线思想；II 真，总线接口按共同总线时钟采样，不要求设备所有内部电路同频；III 真，传统异步握手可分三种互锁方式；IV 假，半同步仍有公共时钟，区别在能否插入等待；V 真，主设备发起访问、从设备响应；VI 在题目简化模型下真，一次传输可分地址与数据阶段；VII 假，单主设备不需要在多个请求主设备间选择。错误 IV、VII 对应 C；A/B 混入 II，D 混入 V，E 混入 VI，故不选；F 不成立。[AHB]

第 22 题 A; “半同步”术语需说明

主题：AHB 通道、等待与时钟。 I 对经典完整 AHB 为真，AHB-Lite 才是单主接口的简化版本；II 假，读数据 HRDATA 和写数据 HWDATA 分开；III 假，AHB 共用地址通道，独立读写地址通道是 AXI 等接口的特征；IV 真，AHB 是同步协议，信号时序以 HCLK 边沿为准。V 在教材“同步时钟加等待应答称为半同步”的分类中可判真，但 ARM 官方仍称它同步协议，HREADY 低只延长数据阶段。VI 真，地址/数据两阶段可以重叠；VII 真，协议不规定电压、驱动强度等电气特性。按 V 的教材分类，错误为 II、III，A 全错；B 含 IV，C 含 V，D 含 VI，E 含 V/VII，不满足题问。若拒绝把同步等待称为半同步而将 V 判假，则 A (II、III) 和 C (III、V) 均是错误陈述的有效组合，不能再保持唯一单选。^[AHB]

第 23 题 E

主题：AHB 突发拍数与每拍大小。 I 假，若“数据块长度”指整个 burst，HSIZE 仅规定每拍大小，HBURST 才规定突发类型/固定拍数；II 真，HTRANS 指出 IDLE、BUSY、NONSEQ、SEQ；III 不能普遍成立，INCR 可为未定长，但 INCR4/8/16 和 WRAP4/8/16 的拍数由 HBURST 指定；IV 真，WRAP4 在边界内循环地址；V 假，INCR8 是 8 拍，总字节数为 $8 \times 2^{\text{HSIZE}}$ ，并非固定 8 B；VI 假，WRAP4 是 4 拍，并非必须 4 个 32 位字。A/B/C/D 都含真的 II 或 IV；E 的 I、V、VI 全为错误，故 E。即使把 III 也判假，也不能以 E 未列出 III 为由否定 E；题目没有要求给出错误陈述的全集。^[AHB]

第 24 题 常规系统假设下 B；一般架构下条件不足

主题：字对齐、位带与可写存储器。 一个 4 B 的对齐 float 起始地址须满足 $A \bmod 4 = 0$ 。III $0\text{xFF}002026$ 和 IV $0\text{x}22002026$ 的低两位为 10_2 ，必不满足；IV 还落在位带别名空间，不能当作普通四字节 float 存储。II、V、VI 都在通用 SRAM 地址区域且字对齐，但该区域是否实际装有 RAM，须由具体芯片决定。I 在通常 STM32 Flash 启动映射中是只读代码/向量映射，不适合保存可写临时变量。在上述常规 Flash 映射与题设假想 SRAM 已实现的条件下，I、III、IV 不可用，B 全部符合；A/C 含可用的 II，D 含 II/V/VI，E 含 V，故 B。可是通用 Cortex-M4 的代码区也允许实现可写 RAM，STM32 也有 SRAM 启动映射；不能仅从“地址为 0”推出架构上绝不可能存放变量。原题未给出完整映射，严格无条件答案不足。^{[ARM-M4][RM0090]}

第 25 题 C

主题：特权与操作模式。 I 假，复位进入特权线程模式；II 假，非特权访问受系统控制空间等限制；III 真，复位使用 MSP；IV 真，有 Thread/Handler 两种模式；V 真，Handler 模式特权；VI 假，Thread 可为特权或非特权。C 的 I、II、VI 全错；A/B 含真的 III/IV，D 含真的 IV/V，E 含真的 IV/V，故 C。^[ARM-M4]

第 26 题 原文无效区间判错时 B、E；明确勘误后 E

主题：内存映射、端序与位带大小。 I 对 Cortex-M3/M4 的内核实现选项而言真，支持大端/小端的集成选择，不代表 STM32 用户可以任意切换端序。II、III 的原文范围 $0\text{x}20000000\text{--}0\text{x}0001\text{FFFF}$ 上界小于下界，无法作为有效地址段判定。IV 假，普通 SRAM 可存放并执行代码；V 真，外部 RAM 可执行代码（受存储属性和 MPU 权限约束）；VI 真，访问受区域属性控制；VII 假，1 MiB 位带区对应 $1\text{MiB} \times 8 \times 4 = 32\text{MiB}$ 别名区，不是 16

MiB。原文 II、III 均把倒置的区间当成有效地址段来陈述；按“无效地址段的这些陈述不成立”处理，II、III、IV、VII 都判错，B（II、III、IV）与 E（III、IV、VII）均满足，原卷不是唯一单选。若把 II/III 视为无法赋予真假的坏题干，则只能报告条件不足，也不能从原文推出唯一 E。

若明确将 II/III 的末地址勘误为 `0x2001FFFF`，则 II 真、III 假，错误为 III、IV、VII。此时 A 含真的 I/V，B 含真的 II，C 含真的 V，D 含真的 VI，E 全错，故 E。直接保留原倒置地址时，不应把这项有条件结论伪装成原题无歧义答案。[ARM-M3][ARM-M4]

第 27 题 字宽入出栈口径 D；任意栈区访问口径 F

主题：双栈与满递减栈。I 真，有 MSP/PSP 两个栈指针；II 若指 SP 与 PUSH/POP、异常自动入栈等字宽栈操作为真，SP 至少字对齐，AAPCS 的公共接口另要求 8 B 对齐；III 假，非特权线程也需要正常使用栈；IV 假，非特权线程并非只能用 MSP；V 假、VI 真，压栈向低地址增长；VII 假，只要可访问且满足属性要求，栈也可位于外部 RAM 等区域。在字宽入出栈语境下，D 的 I、II、VI 全真；A 含 III，B 含 IV，C 含 V，E 含 VII，故 D。若 II 泛指对栈区的任何访问，则可对栈上字节变量执行 `STRB.W R0,[SP,#1]`，该有效地址并非四字对齐，因此 II 为假。此时 A-E 均含假项，只能选 F。SP 对齐规则不禁止对栈内字节变量进行字节访问；不能用该规则直接证明宽泛的 II。[ARM-M4][AAPCS32]

第 28 题 C

主题：存储器映射 I/O。I 假，STM32 外设寄存器采用存储器映射，通过普通 load/store 访问；II、III 真，GPIO IDR/ODR 都是 I/O 寄存器；IV、V、VI 假，定时器、USART、SPI 的控制/状态寄存器地址也属于广义 I/O 端口地址，不能仅把 GPIO 引脚对应寄存器称为端口。A/B 含 I，D 含 V，E 的 V 为假且 VII 在原题根本不存在，只有 C（II、III）全真。[RM0090]

第 29 题 D

主题：向量表与异常抢占。I 假，向量表可通过 VTOR 重定位到合适对齐的 SRAM 等位置；II 真，异常表项通常给出服务程序入口和 Thumb 位；III 假，多个异常可共用同一个默认处理程序地址；IV 假，子优先级只处理同组就绪异常的选择，不赋予抢占能力；V 真，更高组/抢占优先级在未被屏蔽等正常条件下可抢占；VI 假，地址 0 的字是初始 MSP，复位向量在地址 4。D 的 II、V 全真；A/B/C 分别含 I/III/IV，E 含 IV/VI，故 D。[ARM-M4]

第 30 题 E（USART 硬件功能口径）

主题：USART 状态位与多机通信。I 按硬件功能描述判假，基本 USART 按软件设定的波特率工作，不自行与对端协商波特率；II 对 STM32 USART 多机通信模式可判真，地址标记/唤醒机制可使多个接收端按地址区分，不能因为它叫串口就一律判假；III 假，RXNE=1 表示接收数据寄存器非空；IV 假，TXE 对应发送数据寄存器空，移位寄存器及传输完成应看 TC；V、VI 真，分别可启用接收非空和发送寄存器空中断。A 含硬件口径下的 I，B 含 III，C 含 III/IV，D 含 IV，E 的 V、VI 全真。若 I 泛指应用层先在共同波特率上交换协议再切换波特率，则 I 也可实现，此时 A（I、II）也成立；这是上层协议语义，不是硬件自动协商。[RM0090]

三、判断题

第 31 题 ×

主题：系统时钟树。STM32F103 的 SYSCLK 来源是 HSI、HSE 或 PLL 输出；LSI/LSE 主要用于独立看门狗、RTC 等低速用途，不能直接替代 SYSCLK 源。[RM0008]

第 32 题 ×

主题：独立看门狗时钟。IWDG 由独立低速内部振荡器 LSI 驱动，不是高速内部振荡器 HSI；独立性使其在主时钟失常时仍可工作。[RM0008]

第 33 题 ✓

主题：DDR4 预取。标准 DDR4 采用 8n-prefetch。这里的 8 指内部预取宽度相对于每根 DQ 的数据粒度，是每 I/O 的 8 个数据位，并不意味着整片内存数据总线仅有 8 位；x4、x8、x16 器件均可采用 8n 预取。[DDR4]

第 34 题 ✓

主题：SP 对齐。0x20262026 的低两位为 10₂，不是 4 B 对齐。M3 的 SP 低两位固定为 0，不能保持这个精确值；这与该地址是否另有实际 SRAM 是两个不同问题。[ARM-M3][AAPCS32]

第 35 题 ✓

主题：系统控制空间的特权保护。异常优先级寄存器位于受特权保护的系统控制空间，非特权线程不能直接改写/访问这些配置；应经特权服务完成。[ARM-M3]

第 36 题 ✓（实现方法的可行性）

主题：微程序控制。可以把机器指令的操作步骤组织成微指令序列，由微程序控制器执行。题目说“可以”，不是说所有 CPU、尤其所有 Cortex-M 都必须用微程序；硬布线控制器也能实现相同 ISA。

第 37 题 ×

主题：SPLIT 响应与流水化的区别。AHB 两阶段流水线使相邻事务的地址和数据阶段重叠；SPLIT 则是从设备暂时不能完成请求时，允许释放总线给其他主设备，待从设备通知后再仲裁、重试。SPLIT 不是把同一普通传输的地址线 and 数据线分开使用的机制。[AHB]

第 38 题 ✓

主题：指令发射次序与完成次序。不能把“采用按序流水线”扩大成“所有指令结果必严格按序完成”。Cortex-M4F 官方 TRM 明确说明：位于 VDIV 或 VSQRT 之后的纯整数指令可以先完成。例如一条独立整数 ADD 可能在先发射的浮点除法尚未完成时结束。因此题中“不一定先完成”有直接硬件反例，判真。这里引用的是带 FPU 的 Cortex-M4F，恰与 STM32F407 的内核类型相符；不需要借用 Cortex-M7 或把总线写缓冲的外部可见时刻混同

于指令完成。[ARM-M4-TRM][RM0090]

第 39 题 ✓（本题 ARM 32 位 C 数据模型下）

主题：内存映射寄存器与 volatile。若 GPIOA 的 ODR 地址确为 0x4001080C，结构体成员访问最终就是对该地址进行 32 位写；STM32 常用 ABI 中 unsigned long 为 32 位，给出的 volatile 指针解引用表达式表达相同访问。volatile 保留该可观察寄存器访问；它本身不是跨线程同步或内存屏障。[AAPCS32][RM0008]

第 40 题 ✓（通常教材模型）

主题：指令周期与总线周期。一条指令可能经历取指、读取操作数、写回存储器等多次总线传输，因此在传统组成原理模型中，一个指令周期通常包含多个总线周期。命题的“通常”不能改成“一定”：寄存器指令、Cache 命中、流水线重叠时，并不要求每条指令都独占若干外部总线周期。

第 41 题 ×

主题：EXTI 共用中断入口。STM32F407 的 EXTI0–4 有各自入口，而 EXTI5–9 共用 EXTI9_5，EXTI10–15 共用 EXTI15_10。共享 ISR 需检查 pending 位区分来源。[RM0090]

第 42 题 ✓

主题：ISA 与微体系结构。ISA 定义软件可见的指令及状态；微体系结构 μ arch 描述流水线、执行单元、缓存等具体实现。不同 μ arch 可以实现同一 ISA。

第 43 题 ×（“每周期必须独立包含”的严格读法）

主题：仲裁与主设备所有权。多主系统必须解决发生竞争时谁获得总线的问题，但当前主设备可连续保持所有权，仲裁还可与已有传输重叠；每个数据传输周期未必另占一个独立仲裁阶段。因此含“必须包括”的字面强命题不成立。若教材只是把一次完整总线使用过程概括为“申请/仲裁、寻址、传输、结束”等逻辑阶段，则会将此题判真；那是阶段模型的宽松概括，不证明每一周期都重新串行仲裁。速查按严格读法列 ×。[AHB]

第 44 题 ×

主题：位带原子操作。对别名地址写 0/1，由硬件对目标位进行原子更新，可避免软件“读整个字–修改某位–写整个字”的中间竞态，因此题中“仍无法解决”不对。原子位操作并不替代更高层的并发协议，也不能阻止其他代码随后覆盖整个寄存器。[ARM-M4]

第 45 题 ×

主题：内核端序选项与芯片实现。Cortex-M3/M4 的数据存储接口支持小端及字节不变大端的实现选择；具体 STM32 芯片通常固定为小端。题目把 STM32 的实现约束泛化为所有 M3/M4 “只能小端”，故错。[ARM-M3][ARM-M4][RM0090]

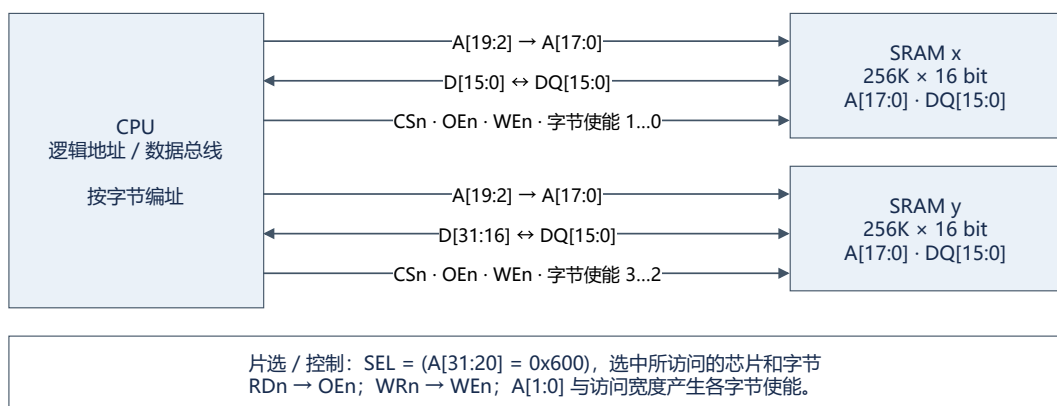
四、设计及综合题

第 46 题 1 MiB；按题设采用两片并联位扩展

主题：容量扩展、字节选通与真实 FSMC。

- (1) 每片 $256\text{K} \times 16\text{bit} = 512\text{KiB}$ ，两片共 1MiB。并联成 $256\text{K} \times 32\text{bit}$ ，恰好一次逻辑字传输包含两片数据。
- (2) 首地址 $0x60000000$ ，容量 $0x100000$ B，末地址为 $0x600FFFFF$ 。
- (3) 两片各有 18 根片内字地址线。32 位逻辑总线以 4 B 为字，CPU A[19:2] 接两片 A[17:0]；CPU A[1:0] 与传输大小用于字节选通，不直接作为片内字地址。接线见图及下表。

2026 年·SRAM 位扩展连接（题设逻辑 32 位总线）



地址范围：0x60000000–0x600FFFFF；所有芯片共享上图所列地址信号。

图中 CSn/OEn/WEn 均低有效；各芯片仅在对对应字节通道被选中时使能。
真实 STM32F103ZE 的 FSMC 数据总线最多 16 位，实际接线方案见正文。

CPU 逻辑信号 连接

A[19:2]	同时连接 x/y 的 A[17:0]。
D[15:0]	SRAM x 的 D[15:0]。
D[31:16]	SRAM y 的 D[15:0]。
A[31:20]	译码等于 0x600 时选中两片（再结合读/写有效条件）。
读/写控制	分别驱动两片 OE/WE；低有效片选经适当反相/门控。
字节使能	x 的低/高字节对应系统字节 0/1, y 对应 2/3；芯片若无字节使能，需附加逻辑或限制为完整字访问。

- (4) 在 STM32 小端约定下，地址 A, A + 1, A + 2, A + 3 分别存 **78, 56, 34, 12**；前两字节来自 x，后两字节来自 y。若重画时交换上下数据半总线，芯片名称也随之交换，不能脱离接线单独断言芯片名。
- (5) 不能对这个外部 RAM 区使用 Cortex-M3 标准位带映射。位带目标区为 **0x20000000** 至 **0x200FFFFF** 的 SRAM 和 **0x40000000** 至 **0x400FFFFF** 的外设，**0x60000000** 不在其中；可用普通读改写、临界区或适用的原子机制另行实现。

题目把 F103ZE 写成具有 32 根外部数据线，以上图是服从该逻辑题设的位扩展答案。真实 STM32F103ZE 的 FSMC 异步 SRAM/NOR 接口只有 8/16 位宽，不能直接把两片并接成一次 32 位外部传输。真实实现可用 16 位数据总线作字扩展：两片共用 D[15:0]，CPU 字节地址 A[18:1] 转

成片内 A[17:0]，用更高位和译码选择 x/y；连续 1 MiB 需要适当外部译码或映射设计。32 位处理器读字分成两个 16 位传输；若 x 占低 512 KiB，则基地址第一个字的四个字节均来自 x。不能把此真实 16 位方案的第（4）问答案混入上面的并联 32 位方案。[DS-F103][RM0008][ARM-M3]

第 47 题 4 组、每组 16 行；两个程序命中率均为 63/64

主题：组相联映射、容量匹配与访问次序。

- (1) Cache 总行数 $16\text{KiB}/256\text{B} = 64$ ，16 路意味着每组 16 行，故组数 $64/16 = 4$ 。地址分解为 20 位主存地址中的 8 位行内偏移、2 位组号和 10 位标记。
- (2) 若沿用组相联教材的“页”为覆盖全部组号的一轮主存块，页大小 $4 \times 256 = 1024\text{B}$ ，主存共 $2^{20}/2^{10} = 1024$ 页，每页 4 个主存块，可分别映射到 4 组。此处主存中称“块”更准确，Cache 中称“行”。题目未给操作系统页大小；若另一教材把“页”规定为整个 Cache 容量 16 KiB，则是 64 页、每页 64 块。二者是不同分组定义，不能与标记位数混算。本解析采用前一种、与 10 位 tag 对应的定义。
- (3) 数组容量 $64 \times 64 \times 4 = 16384\text{B}$ ，正好等于数据 Cache。数组从 0 开始对齐，64 个行块按组号模 4 均分，每组恰有 16 块，与 16 路相等。因此不仅总容量够，逐组容量也够，可同时全部保留。
- (4) 元素地址为 $(1 \times 64 + 31) \times 4 = 380 = 0x17C$ ，占 $0x17C--0x17F$ ；对应主存块为 $0x100--0x1FF$ 。块号 1 映射到组 $1 \bmod 4 = 1$ ，行内偏移 $0x7C$ ，但能放在该组任意一路，题目未给替换/初始分配规则，不能唯一确定 Cache 物理行。
若人为按“组优先、组内连续放 16 路”编号 Cache 字节，组 1 总范围是 $0x1000--0x1FFF$ ；具体路 $w \in [0, 15]$ 的行范围为 $0x1000 + 256w$ 至 $0x10FF + 256w$ 。若采用“路优先”编号，同组各路行基址则为 $0x100 + 0x400w$ 。Cache 内部编号不是 CPU 可访问的另一段主存地址，两个编号方法描述同一映射。
- (5) 每次运行都从空 Cache 开始，仅统计数组的 $64^2 = 4096$ 次读取。程序 1 顺行读取，每行首元素 1 次强制未命中，随后 63 次命中，共 64 次未命中。程序 2 第一列依次触及所有 64 行，产生 64 次未命中；此时每组正好占满 16 路而无冲突淘汰，以后 63 列全部命中。因此

$$h_1 = h_2 = \frac{4096 - 64}{4096} = \frac{63}{64} = 98.4375\%.$$

列优先访问并不自动意味着零命中；关键是本题整个工作集正好装入 Cache 且逐组不溢出。

原程序 2 缺少 `return sum`；且与程序 1 同名，不能不加说明地当作同一 C 文件中的两个正确函数。题目将其称为伪代码，本题 Cache 结果按其循环内的数组访问序列计算；实际 C 代码应将第二函数改名为 `sum_array2` 并补上返回语句。

第 48 题 示例：PSC=7999，ARR=1999，CCR3=500

主题：定时器分频、低电平点灯与 PWM。

- (1) ①使能 AHB1 上 GPIOE 外设时钟；②使能 APB1 上 TIM2 时钟；③把 PB10 的复用选择设为 TIM2，对应 TIM2_CH3，但它不能代替 `GPIO_Mode_AF` 配置；④设置 `TIM_CKD_DIV1`，令采样/数字滤波的 $t_{DTS} = t_{CK_INT}$ ，不是把 PSC 再乘一个用于基本计数的分频因子。
[RM0090]

- (2) 题设定定时器输入为 8 MHz。向上计数含 0 至 ARR 共 ARR+1 个计数，故

$$T = \frac{(PSC + 1)(ARR + 1)}{8 \times 10^6} = 2 \text{ s.}$$

可令 PSC=7999，使计数频率为 1 kHz，再令 ARR=1999，得到 2000 个计数、2 s 周期。因此④填 7999、⑤填 1999。这不是唯一组合：还可选取乘积为 16×10^6 、ARR+1 可被 4 整除且寄存器取值合法的整数组合，使比较值 (ARR + 1)/4 也是整数。

- (3) 图中 LED0 从 VCC 经电阻/LED 接向 PB10，PB10 为低时灌电流点亮。PWM1、向上计数、正极性下，CNT < CCR3 时 PB10 高（灯灭），其他时刻低（灯亮）。灭 0.5 s 对应高电平 500 个计数，亮 1.5 s 对应低电平 1500 个计数，故 my_ccr3_value=500，不是 1500。一般式为 CCR3 = (ARR + 1)/4。

给定代码实际问题。 GPIOB 配置缺失 GPIO_Mode；若结构体静态零初始化则可能保持输入模式，若未初始化则值不确定。GPIOE 的 GPIO_Mode=GPIO_OType_PP 混用了枚举，不能保证配置成输出。输出比较结构体未完整初始化，且注释写 CCR1，而调用实际针对通道 3。只填④⑤⑦不能保证原程序正确驱动 PB10。下面给出问题时钟条件下的关键可运行修正版（使用 STM32F4 标准外设库，时钟树已按题设配置）：

```

1 void LED0_PWM_Init(void)
2 {
3     GPIO_InitTypeDef g;
4     TIM_TimeBaseInitTypeDef t;
5     TIM_OCInitTypeDef oc;
6     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB, ENABLE);
7     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
8     GPIO_StructInit(&g);
9     GPIO_PinAFConfig(GPIOB, GPIO_PinSource10, GPIO_AF_TIM2);
10    g.GPIO_Pin = GPIO_Pin_10;
11    g.GPIO_Mode = GPIO_Mode_AF;
12    g.GPIO_OType = GPIO_OType_PP;
13    g.GPIO_Speed = GPIO_Speed_100MHz;
14    g.GPIO_PuPd = GPIO_PuPd_UP;
15    GPIO_Init(GPIOB, &g);
16    TIM_Cmd(TIM2, DISABLE);
17    TIM_TimeBaseStructInit(&t);
18    t.TIM_Prescaler = 7999;
19    t.TIM_Period = 1999;
20    t.TIM_CounterMode = TIM_CounterMode_Up;
21    t.TIM_ClockDivision = TIM_CKD_DIV1;
22    TIM_TimeBaseInit(TIM2, &t);
23    TIM_OCStructInit(&oc);
24    oc.TIM_OCMode = TIM_OCMode_PWM1;
25    oc.TIM_OutputState = TIM_OutputState_Enable;
26    oc.TIM_OCPolarity = TIM_OCPolarity_High;
27    oc.TIM_Pulse = 500;
28    TIM_OC3Init(TIM2, &oc);
29    TIM_OC3PreloadConfig(TIM2, TIM_OCPreload_Enable);
30    TIM_ARRPreloadConfig(TIM2, ENABLE);
31    TIM_SetCounter(TIM2, 0);
32    TIM_GenerateEvent(TIM2, TIM_EventSource_Update);
33    TIM_ClearFlag(TIM2, TIM_FLAG_Update);

```

```

34     TIM_Cmd(TIM2, ENABLE);
35 }

```

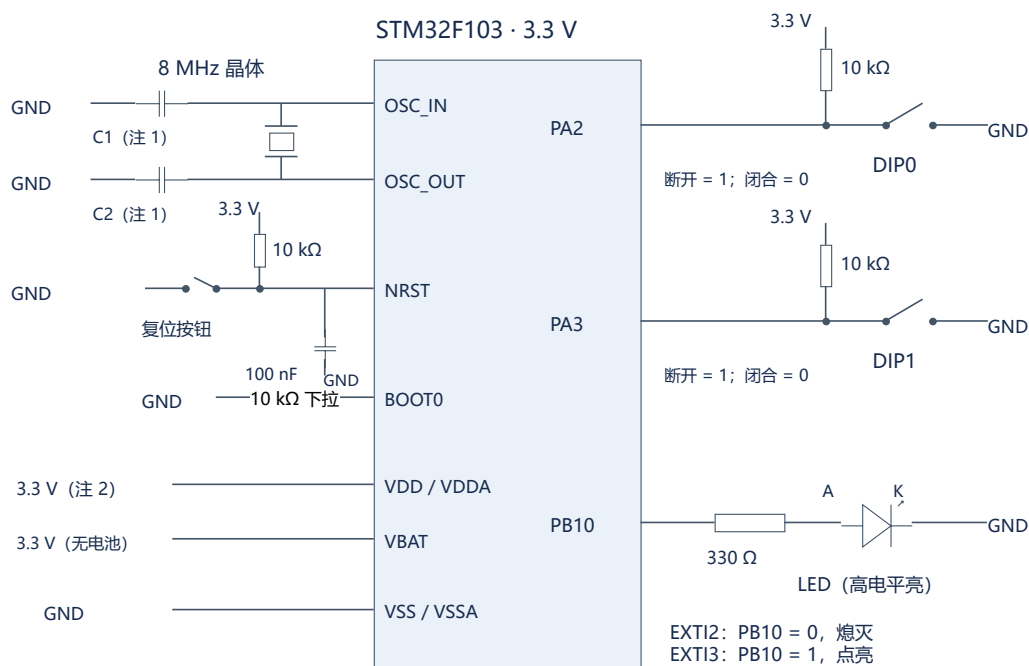
此修正版只驱动 LED0，未使用的 PE5/LED1 无须初始化；若同时使用 LED1，应独立设 GPIOE 为 GPIO_Mode_OUT。按题面是固定亮灭的 0.5 Hz 闪烁；通常所谓“呼吸灯”还需缓慢改变 PWM 占空比，原题未要求该渐变控制。

第 49 题 PA2/PA3 下降沿中断；PB10 高电平点灯

主题：最小系统、EXTI 路由与抢占优先级。

- (1) DIP0 接 PA2，DIP1 接 PA3，两输入各上拉至 3.3 V，拨动闭合接地产生下降沿。③写 BSRR 的 bit 26，把 PB10 清为 0，而题意称此时 LED 熄灭，所以 LED 必须按 PB10 高电平有效连接：PB10 经限流电阻和正向 LED 接地。不是照搬第 48 题低电平点亮的 LED0 接法。

2026 年第 49 题·STM32F103 最小系统与 I/O 连接



注 1：晶体负载电容按晶体 CL 和板级寄生电容选取，不能仅凭题文唯一给定。
 注 2：每个 VDD 引脚附近放置 100 nF 去耦；VDDA 另放 100 nF + 1 μF，公共电源配储能电容。
 BOOT0 = 0 从主 Flash 启动；未使用的 BOOT1/PB2 不悬空。本图为连接示意，布局需遵守器件手册。

最小系统包括正确的 VDD/VDDA、VSS/VSSA 供电与去耦，NRST 上拉及接地复位键，HSE 晶体跨接 OSC_IN/OSC_OUT 及匹配负载电容，BOOT0 拉低从 Flash 启动。具体电源脚、振荡器脚编号按选定封装核对；不把所有封装的 OSC 引脚强行等同于 PD0/PD1。^[DS-F103]

- (2) ①填 EXTI_Line2，即位掩码 0x00000004；后面 PA3 配置相应填 EXTI_Line3，即 0x00000008。这里是 EXTI 线路掩码，不是 NVIC 的 IRQ 编号。
 (3) ②填 EXTI2_IRQn（数值 8）；第二组 NVIC 配置填 EXTI3_IRQn（数值 9）。IRQn 不是向量表地址，也不是 EXTI 的位掩码。
 (4) 0x04000000 = 1 << 26，是 BSRR 高半字的 BR10。写 1 使 GPIOB 的 ODR bit 10 清零，

其他位保持不变；因此 PB10 输出低，按本题接法熄灯。它不是在 PB26 输出高电平。
[RM0008]

- (5) ④填 `GPIOB->BSRR = 0x00000400;`，即设置低半字 BS10，令 PB10 高电平点灯；之后仍须保留原有清除 `EXTI_Line3 pending` 的调用。
- (6) 配置正确并使能后可能发生嵌套：EXTI2 抢占优先级 0 高于 EXTI3 的 1，因而 DIP0 事件可打断正在执行的 EXTI3 ISR；反方向不行。两者子优先级均为 1 不妨碍不同抢占级间的抢占。若从未在 ISR 执行期间到达更高优先级事件，则没有实际发生嵌套，所以不应写成“每次必然嵌套”。

给定代码实际问题与修正版。原题称 F103，却调用 F4 系列的 SYSCFG 相关库接口；它在 F103 标准外设库中不能按原样编译。应使能 AFIO 并用 `GPIO_EXTILineConfig`。以下给出 F103 的配置和 ISR 核心实现；其余头文件、启动文件和时钟初始化使用对应 F103 工程，DIP 配有题图上拉，并未加入题外的按键消抖逻辑。

```

1 void DIP_LED_Init(void)
2 {
3     GPIO_InitTypeDef g;
4     EXTI_InitTypeDef e;
5     NVIC_InitTypeDef n;
6     RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO |
7         RCC_APB2Periph_GPIOA | RCC_APB2Periph_GPIOB, ENABLE);
8     GPIO_StructInit(&g);
9     g.GPIO_Pin = GPIO_Pin_2 | GPIO_Pin_3;
10    g.GPIO_Mode = GPIO_Mode_IPU;
11    GPIO_Init(GPIOA, &g);
12    GPIOB->BSRR = 1UL << 26; // LED initially off
13    g.GPIO_Pin = GPIO_Pin_10;
14    g.GPIO_Mode = GPIO_Mode_Out_PP;
15    g.GPIO_Speed = GPIO_Speed_2MHz;
16    GPIO_Init(GPIOB, &g);
17    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource2);
18    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource3);
19    EXTI_StructInit(&e);
20    e.EXTI_Line = EXTI_Line2 | EXTI_Line3;
21    e.EXTI_Mode = EXTI_Mode_Interrupt;
22    e.EXTI_Trigger = EXTI_Trigger_Falling;
23    e.EXTI_LineCmd = ENABLE;
24    EXTI_ClearITPendingBit(EXTI_Line2 | EXTI_Line3);
25    EXTI_Init(&e);
26    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
27    n.NVIC_IRQChannel = EXTI2_IRQn;
28    n.NVIC_IRQChannelPreemptionPriority = 0;
29    n.NVIC_IRQChannelSubPriority = 1;
30    n.NVIC_IRQChannelCmd = ENABLE;
31    NVIC_Init(&n);
32    n.NVIC_IRQChannel = EXTI3_IRQn;
33    n.NVIC_IRQChannelPreemptionPriority = 1;
34    NVIC_Init(&n);
35 }
36 void EXTI2_IRQHandler(void)
37 {

```

```
38     if (EXTI_GetITStatus(EXTI_Line2) != RESET) {
39         GPIOB->BSRR = 1UL << 26;
40         EXTI_ClearITPendingBit(EXTI_Line2);
41     }
42 }
43 void EXTI3_IRQHandler(void)
44 {
45     if (EXTI_GetITStatus(EXTI_Line3) != RESET) {
46         GPIOB->BSRR = 1UL << 10;
47         EXTI_ClearITPendingBit(EXTI_Line3);
48     }
49 }
```

以上分组利用 F103 实现的 4 个优先级位: Group 1 提供 1 位抢占级和 3 位子优先级。代码中的 0/1 抢占级与子级 1 均合法; 所给值是逻辑优先级, 库函数负责移位编码。^{[RM0008][ARM-M3]}

核验资料

以下为解析中使用的官方资料。文内方括号标记可点击跳转至本页，资料标题可点击打开来源。公式、数值和 Cache 访问统计由题设独立推导；官方手册用于核验架构及器件行为。查阅日期为 2026 年 9 月 21 日。

ARM-M3

Arm, *Cortex-M3 Devices Generic User Guide*, DUI 0552A。核验程序员模型、CONTROL/SP、异常向量、存储器属性、位带及非对齐访问。

ARM-M4

Arm, *Cortex-M4 Devices Generic User Guide*, DUI 0553B。核验复位向量 bit 0、特权级、位带映射、异常及指令访问行为。

ARM-M4-TRM

Arm, *Cortex-M4 Technical Reference Manual*, DDI 0439B。第 7.2.3 节核验 Cortex-M4F 浮点除法 / 平方根指令与后续独立整数指令可不按发起顺序完成；这与按序发射并不矛盾。

RM0090

STMicroelectronics, *STM32F405/415、F407/417、F427/437、F429/439 Reference Manual*, RM0090。重点查阅 RCC、GPIO、SYSCFG/EXTI、通用定时器与 USART 章节。

RM0008

STMicroelectronics, *STM32F101/102/103/105/107 Reference Manual*, RM0008。重点查阅存储器映射、AFIO、GPIO、EXTI、FSMC、RCC、IWDG 与通用定时器。

AHB

Arm, *AMBA Specification (Rev 2.0)*, IHI 0011A。第 1.9 节说明电气特征的范围，第 2.2 节列 AHB 信号，第 3 章说明流水传输、突发、仲裁和 SPLIT，第 5 章说明 APB。涉及 SPLIT 的题目应以完整 AHB 为背景，而非删去此机制的 AHB-Lite。

AAPCS32

Arm, *Procedure Call Standard for the Arm Architecture*。核验 ARM 32 位 C 数据模型、基本数据类型与栈对齐要求。

DS-F103

STMicroelectronics, *STM32F103xC/xD/xE Datasheet*, DS5792。核验 FSMC 外部数据宽度、OSC_IN/OSC_OUT 引脚、供电、复位与外部时钟连接。

ARM-R5

Arm, *Addressing functional safety applications with Arm Cortex-R5*。核验双核锁步的故障检测用途。

STM-F1-CMSIS

STMicroelectronics, STM32F103 器件头文件。核验 GPIO 寄存器结构、地址与中断号。适用代码寄存器比较，不据此替代具体封装的数据手册。

STM-F4-CMSIS

STMicroelectronics, STM32F407 器件头文件。核验 F4 GPIO 寄存器布局及与 F1 的差异。

ARM-TCM

Arm, *Cortex-R Series Programmer's Guide*, DEN0042A，第 8.1 节。用于澄清 TCM 作为物理地址图中真实存储的含义；此引用不表示 STM32F407 具有文中 Cortex-R 的 TCM 实现。

DDR4

Samsung, *DDR4 SDRAM Device Operation*, Rev. 1.1, 第 1.2 节。核验 DDR4 的 8n 预取结构及外部双沿数据传输。

核验范围

本册对全部题号与设计题子问进行覆盖检查，对进制、浮点、补码溢出、地址边界和 Cache 访问结果作独立数值核验，并对导出的 PDF 逐页检查显示效果。涉及实验箱型号、原卷未给出的电路细节及授课术语，无法仅凭试卷确定的部分已在对应题目标明。代码以静态分析和官方寄存器定义核验；未声称完成实验箱实测测试。